

Slides licensed under Creative Commons
Attribution [CC-BY 4.0](https://creativecommons.org/licenses/by/4.0/), unless noted otherwise

Image processing

Assaf B. Spanier

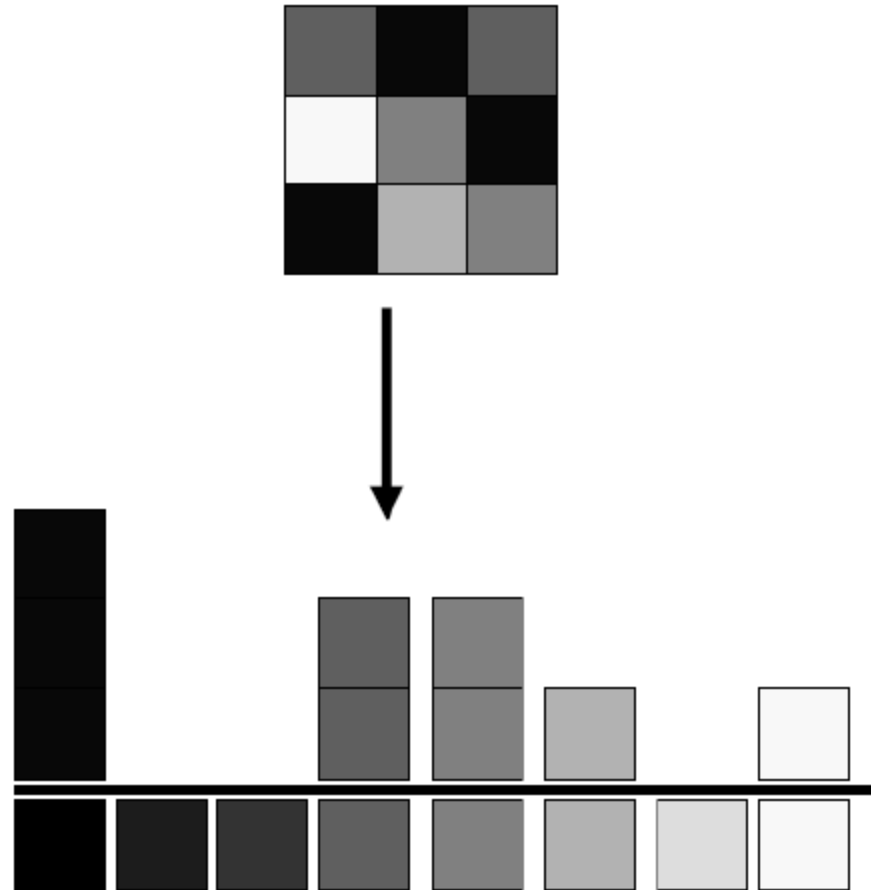
עיבוד תמונה – שיעור 3

- 1. חזרה
- 2. Convolutions \ קונבולציה
 - 1. הגדרה
 - 2. דוגמאות ראשונות
- 3. מה עושים עם קונבולציה – חלק א'
 - 1. Blur
 - 1. אחיד
 - 2. גאוסיאני
 - 3. Bi-lateral filter
 - 2. Edges
 - 1. First derivative

עיבוד תמונה – שיעור 3

- 1. חזרה
- 2. Convolutions \ קונבולציה
 - 1. הגדרה
 - 2. דוגמאות ראשונות
 - 3. מה עושים עם קונבולציה – חלק א'
 - 1. Blur
 - 1. אחיד
 - 2. גאוסיאני
 - 3. Bi-lateral filter
 - 2. Edges
 - 1. First derivative

A ^{תזרה} s_i mage and its histogram





ההסטוגרמה - בצורה פורמלית:

- אנו עוסקים בתמונות אפור
- כל פיקסל הוא מספר בתחום 0 עד 1
- נחלק את הקטע $[0,1]$ ל- L חתיכות קטנות ($L=256$)
- נסמן את מרכזי הקטעים (שהם רמות אפור) כ- $r_0, r_1, \dots, r_{L-1}$
- נסמן את מספר הפיקסלים שרמת האפור שלהם bin ה- r_k כ- n_k .
- ההיסטוגרמה של התמונה היא פונקציה המוגדרת כך: $p(r_k) = \frac{n_k}{n}$

זה למעשה
ההסתברות
לפיקסל שרמת
האפור r_k

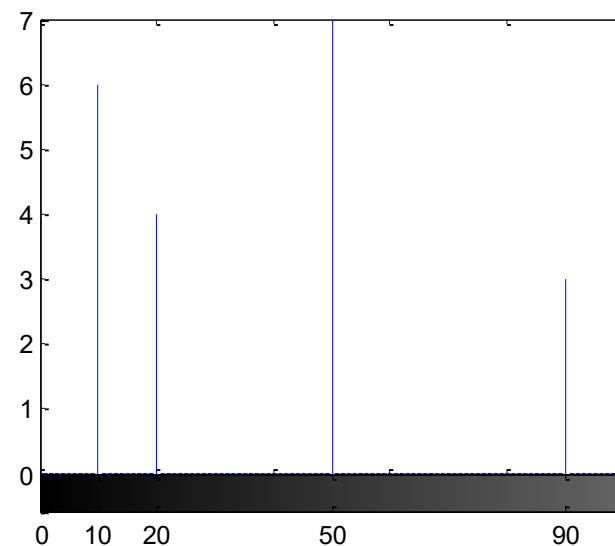
- כאשר n הוא מספר הפיקסלים בתמונה.



היסטוגרמה – דוגמא נוספת

A graph indicating the number of times each gray level occurs in the image

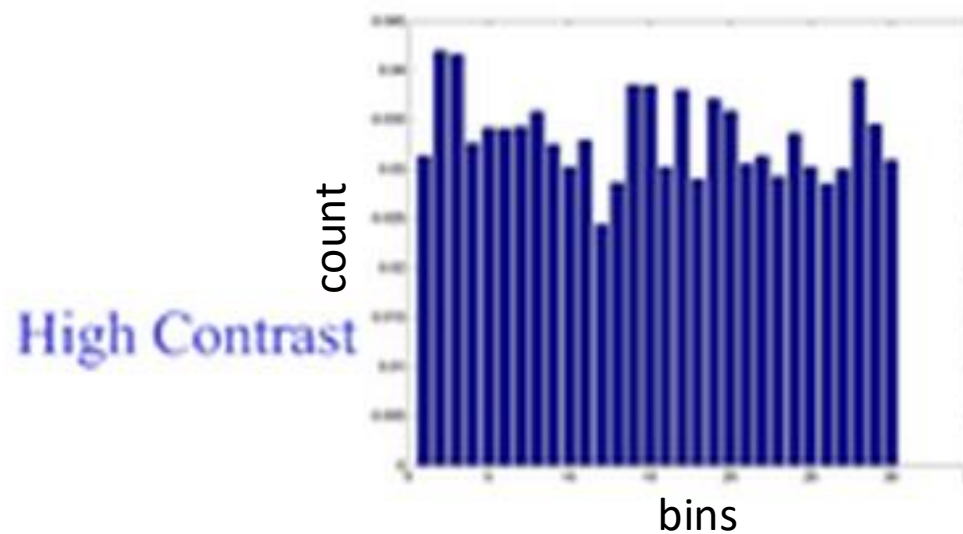
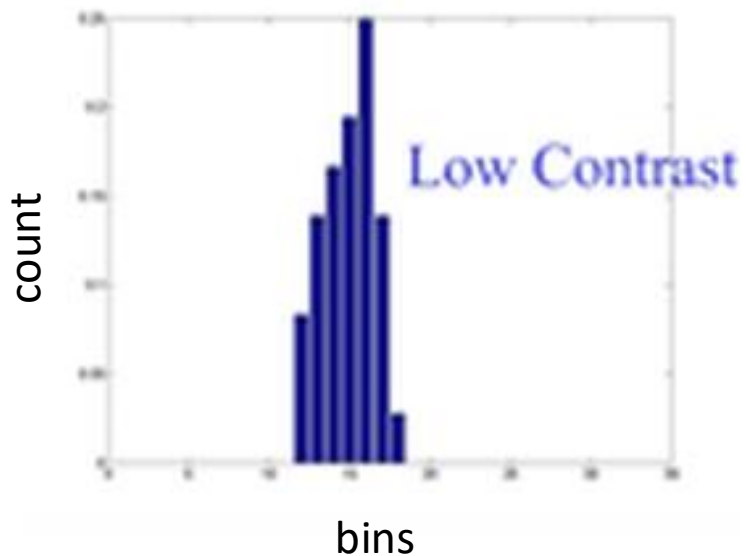
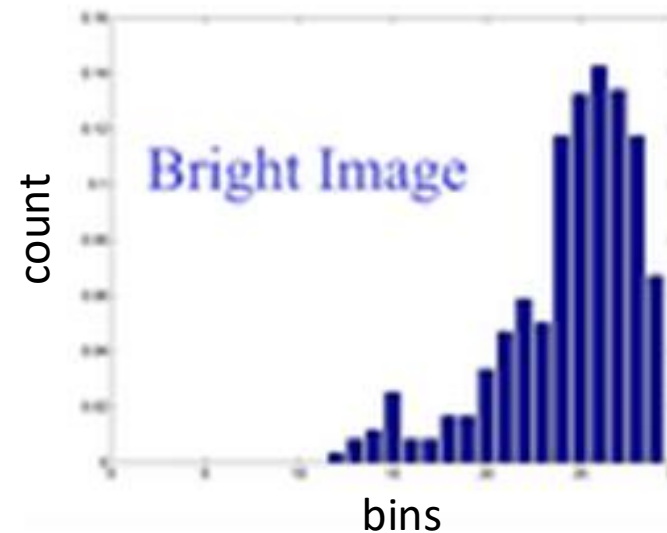
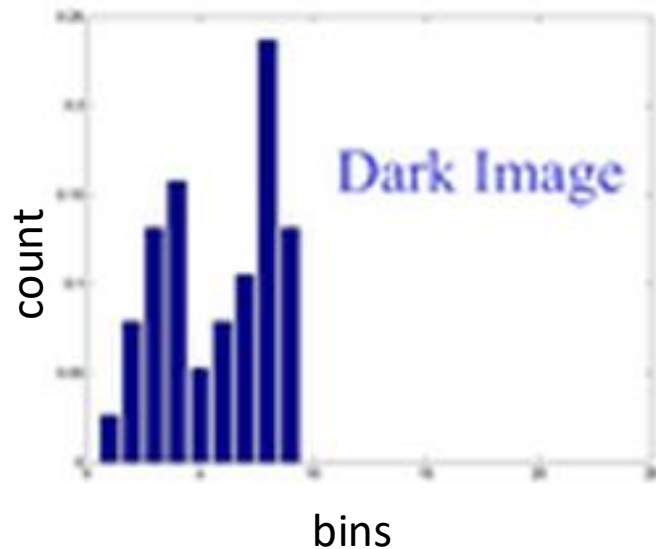
```
a = [10 10 10 10 10;  
      20 20 20 20 10;  
      50 50 50 50 50;  
      90 90 90 50 50]
```





שג

Histogram Types

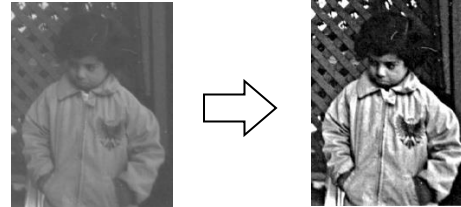


עיבוד תמונה – שיעור 3

- 1. חזרה
- 2. Convolutions \ קונבולציה
 - 1. הגדרה
 - 2. דוגמאות ראשונות
 - 3. מה עושים עם קונבולציה – חלק א'
 - 1. Blur
 - 1. אחיד
 - 2. גאוסיאני
 - 3. Bi-lateral filter
 - 2. Edges
 - 1. First derivative

Spatial operator defined on a neighborhood

- Application



- Method

$$f \xrightarrow{T_N(.)} g = T_N(f)$$

$$f(x,y) \quad , \quad 1 \leq x \leq M, 1 \leq y \leq N$$

$$g(x,y) \quad , \quad 1 \leq x \leq M, 1 \leq y \leq N$$

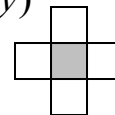
$T_N(.)$: Spatial operator defined on a neighborhood N of a given pixel

$N_0(x,y)$

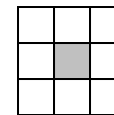


point processing

$N_4(x,y)$



$N_8(x,y)$



***mask/kernel
processing***

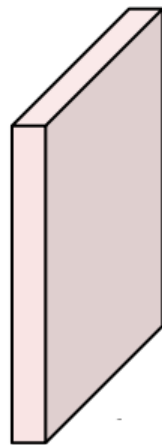
פעולות במרחב התמונה

Filter convolutional

פעולות במרחב התמונה

Filter convolutional

תמונה

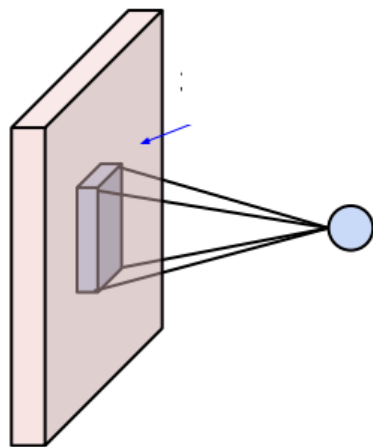


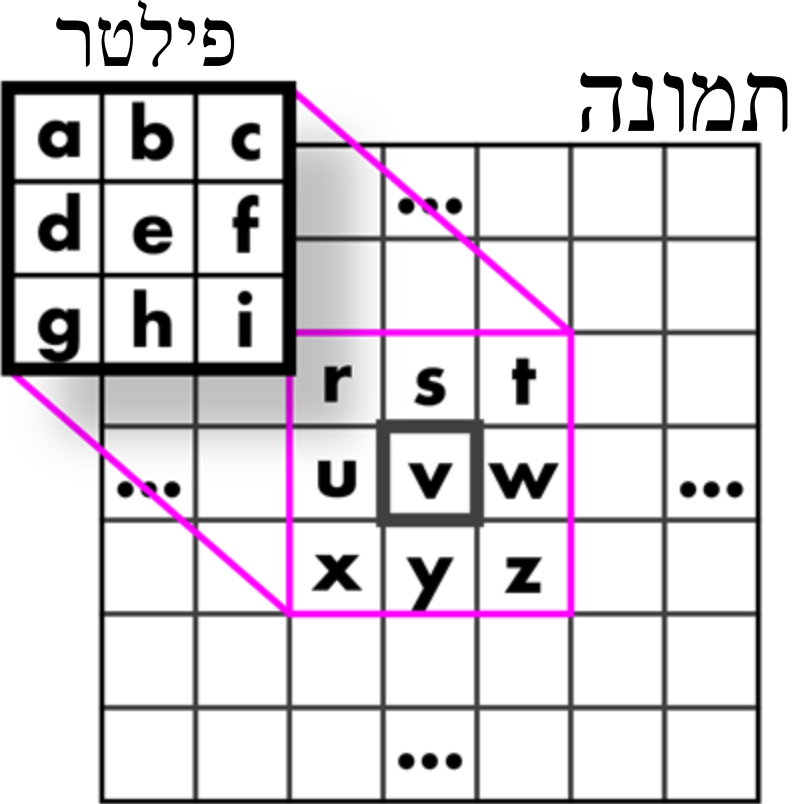
פילטר



פעולות במרחב התמונה

Filter convolutional





$$\mathbf{q} = \mathbf{a} \times \mathbf{r} + \mathbf{b} \times \mathbf{s} + \mathbf{c} \times \mathbf{t} + \mathbf{d} \times \mathbf{u} + \mathbf{e} \times \mathbf{v} + \mathbf{f} \times \mathbf{w} + \mathbf{g} \times \mathbf{x} + \mathbf{h} \times \mathbf{y} + \mathbf{i} \times \mathbf{z}$$

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Kernel Matrix		
0	-1	0
-1	5	-1
0	-1	0

320				

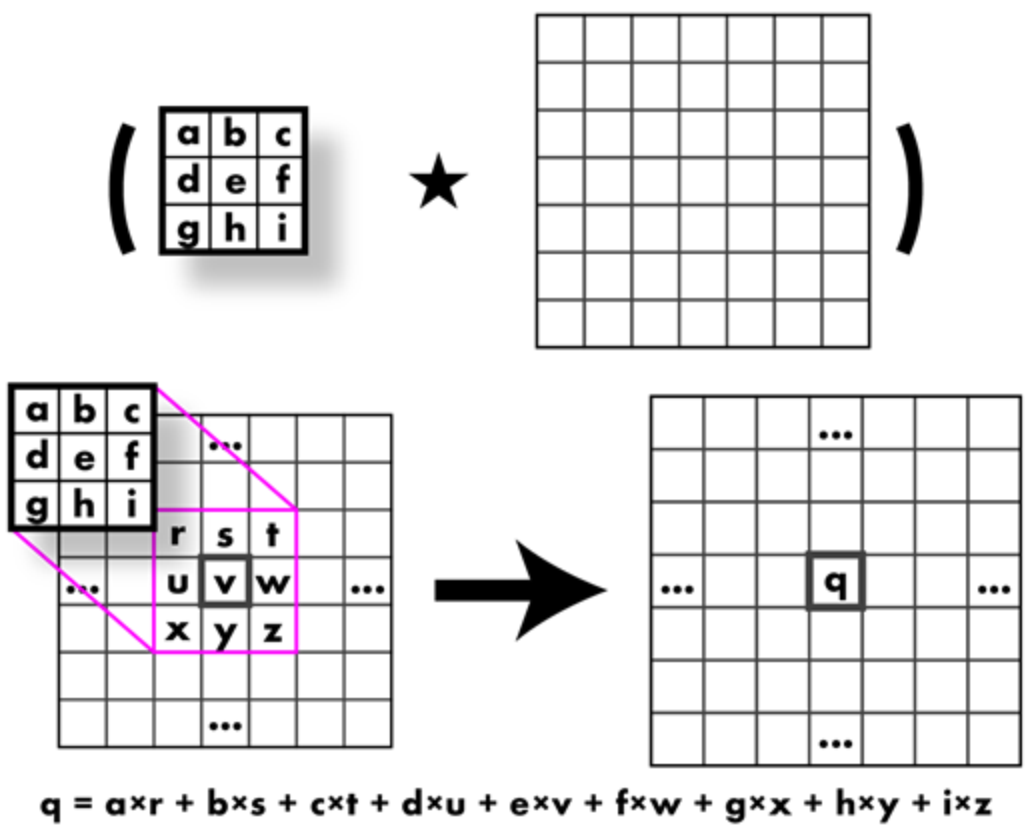
Output Matrix

$$\begin{aligned}
 &0 * 0 + 0 * -1 + 0 * 0 \\
 &+ 0 * -1 + 105 * 5 + 102 * -1 \\
 &+ 0 * 0 + 103 * -1 + 99 * 0 = 320
 \end{aligned}$$

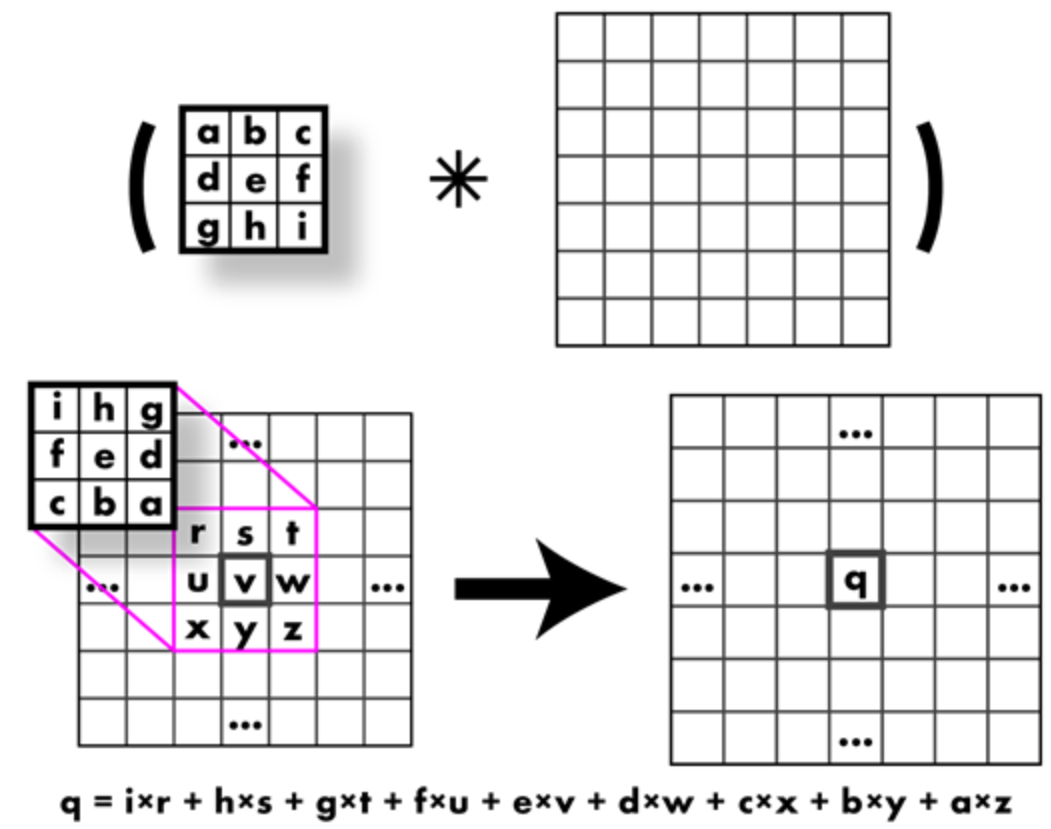
Convolutions with zero padding

Cross-Correlation vs Convolution

Cross-Correlation



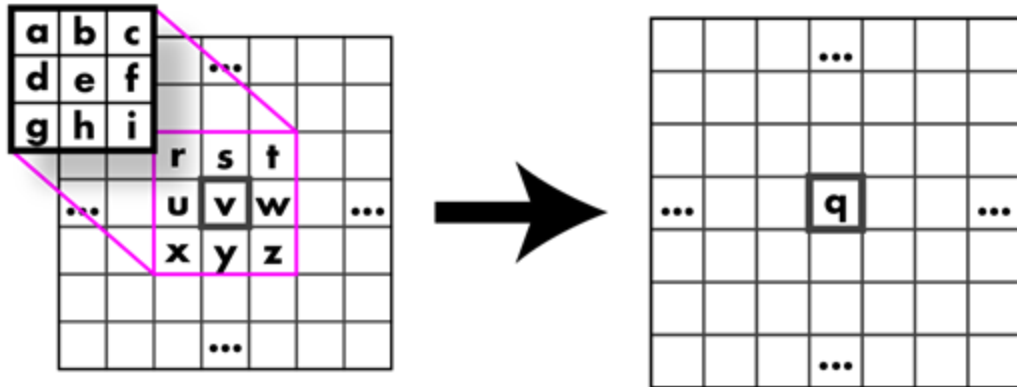
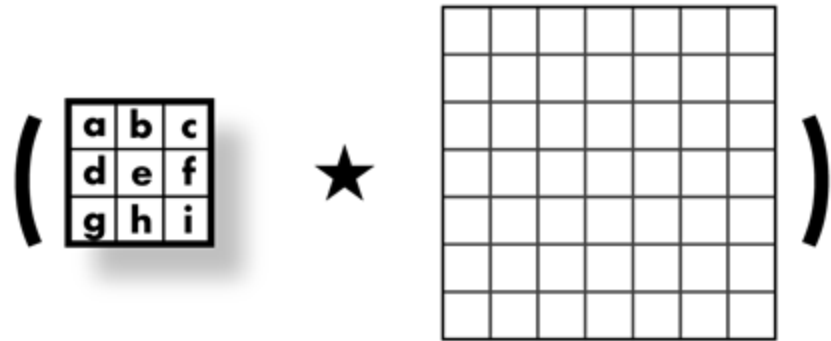
Convolution



Cross-Correlation vs Convolution

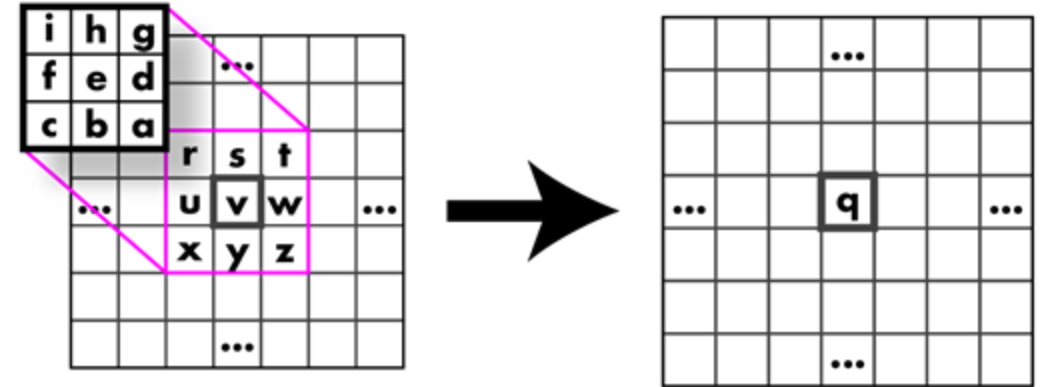
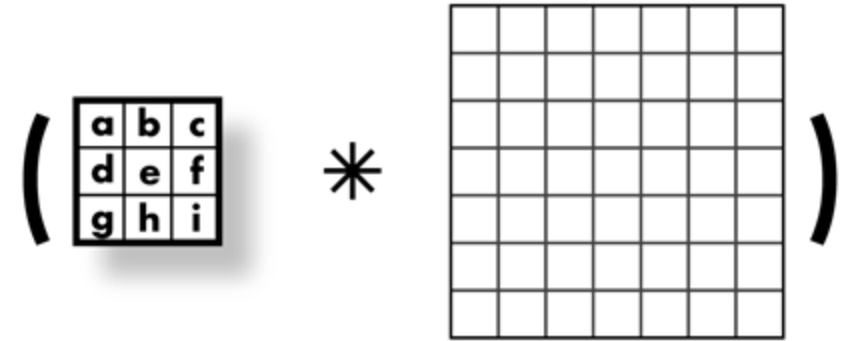
Do this!

Cross-Correlation



$$q = a \times r + b \times s + c \times t + d \times u + e \times v + f \times w + g \times x + h \times y + i \times z$$

Convolution

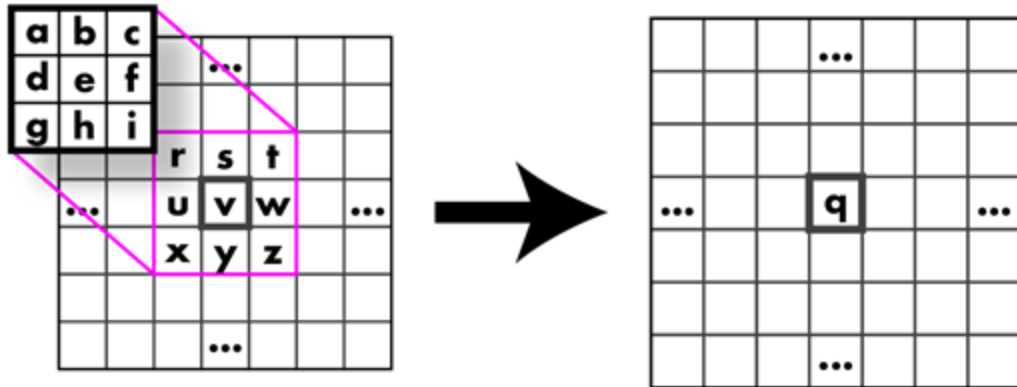


$$q = i \times r + h \times s + g \times t + f \times u + e \times v + d \times w + c \times x + b \times y + a \times z$$

Cross-Correlation vs Convolution

Cross-Correlation

$$\left(\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \star \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix} \right)$$

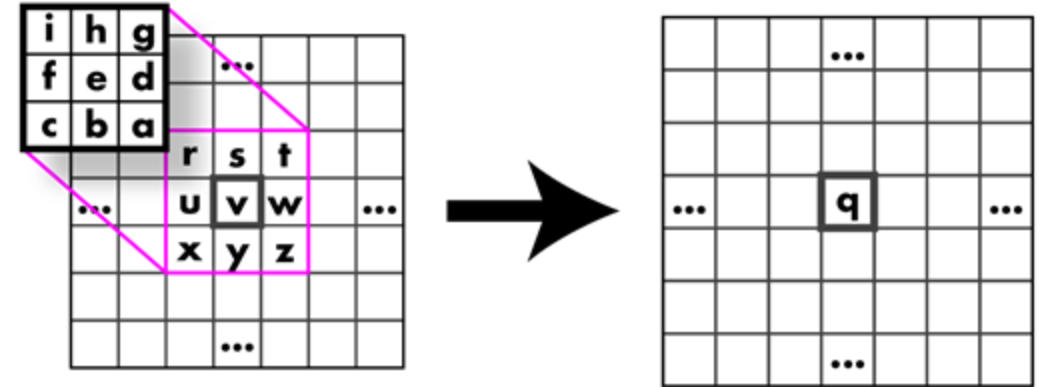


$$q = a \times r + b \times s + c \times t + d \times u + e \times v + f \times w + g \times x + h \times y + i \times z$$

These are nicer mathematically

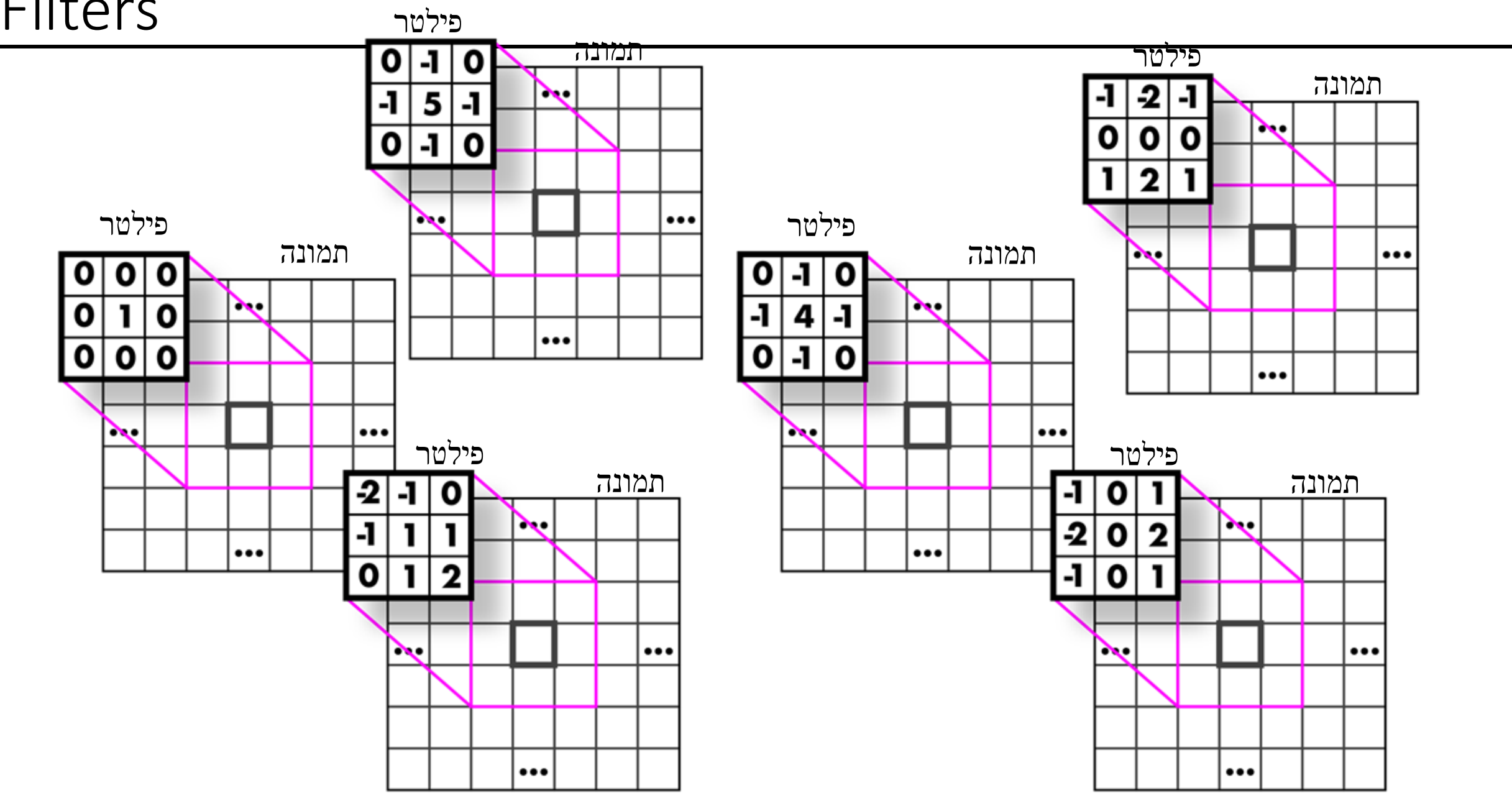
Convolution

$$\left(\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} * \begin{bmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{bmatrix} \right)$$



$$q = i \times r + h \times s + g \times t + f \times u + e \times v + d \times w + c \times x + b \times y + a \times z$$

Filters



עיבוד תמונה – שיעור 3 (שבוע 5)

- 1. חזרה
- 2. Convolutions \ קונבולציה
 - 1. הגדרה
 - 2. דוגמאות ראשונות
- 3. מה עושים עם קונבולציה – חלק א'
 - 1. Blur
 - 1. אחיד
 - 2. גאוסיאני
 - 3. Bi-lateral filter
 - 2. Edges
 - 1. First derivative

Practice with linear filters

תמונה



Original

פילטר

0	0	0
0	1	0
0	0	0

?

- [menti.com](https://www.menti.com)

Practice with linear filters

תמונה



Original

פילטר

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Practice with linear filters

תמונה



Original

פילטר

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

So what can we do with these convolutions anyway?

Mathematically: all the nice things

- Commutative
 - $A * B = B * A$
- Associative
 - $A * (B * C) = (A * B) * C$
- Distributes over addition
 - $A * (B + C) = A * B + A * C$
- Plays well with scalars
 - $x(A * B) = (xA) * B = A * (xB)$

So what can we do with these convolutions anyway?

This means some convolutions decompose:

- 2d gaussian is just composition of 1d gaussians
 - Faster to run 2 1d convolutions

עיבוד תמונה – שיעור 3

- 1. חזרה
- 2. Convolutions \ קונבולציה
 - 1. הגדרה
 - 2. דוגמאות ראשונות
 - 3. מה עושים עם קונבולציה – חלק א'
- 1. Blur
 - 1. אחיד
 - 2. גאוסיאני
 - 3. Bi-lateral filter
- 2. Edges
 - 1. First derivative

עיבוד תמונה – שיעור 3

- Blurring
 - 1. חזרה
 - 2. Convolutions \ קונבולציה
- Edges
 - 1. הגדרה
 - 2. דוגמאות ראשונות
- Sharpening
 - 3. מה עושים עם קונבולציה – חלק א'
- Features
 - 1. Blur
- Derivatives
 - 1. אחיד
 - 2. גאוסיאני
 - 3. Bi-lateral filter
- Classification
 - 2. Edges
- Detection
 - 1. First derivative
- Image captioning
- ...

עיבוד תמונה – שיעור 3

- Blurring
- Edges
- Sharpening
- Features
- Derivatives
- Super-resolution
- Classification
- Detection
- Image captioning
- ...

- .1 חזרה
- .2 Convolutions \ קונבולציה
- .1 הגדרה
- .2 דוגמאות ראשונות
- .3 מה עושים עם קונבולציה – חלק א'
- .1 Blur
- .1 אחיד
- .2 גאוסיאני
- .3 Bi-lateral filter
- .2 Edges
- .1 First derivative

All of computer vision
is **convolutions**
(basically)

עיבוד תמונה – שיעור 3

- Blurring

- Edges
- Sharpening
- Features
- Derivatives
- Super-resolution
- Classification
- Detection
- Image captioning
- ...

- .1 חזרה
- .2 Convolutions \ קונבולציה
 - .1 הגדרה
 - .2 דוגמאות ראשונות
 - .3 מה עושים עם קונבולציה – חלק א'
 - .1 Blur
 - .1 אחיד
 - .2 גאוסיאני
 - .3 Bi-lateral filter
 - .2 Edges
 - .1 First derivative

עיבוד תמונה – שיעור 3

- Blurring

- Edges
- Sharpening
- Features
- Derivatives
- Super-resolution
- Classification
- Detection
- Image captioning
- ...

- .1 חזרה
- .2 Convolutions \ קונבולציה
 - .1 הגדרה
 - .2 דוגמאות ראשונות
 - .3 מה עושים עם קונבולציה – חלק א'
 - .1 Blur
 - .1 אחיד
 - .2 גאוסיאני
 - .3 Bi-lateral filter
 - .2 Edges
 - .1 First derivative

Image filtering

התמונה

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$

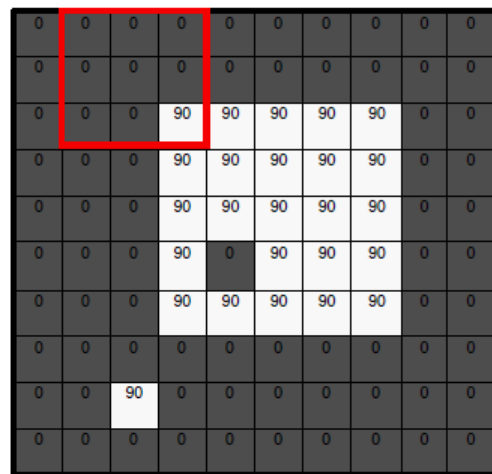
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

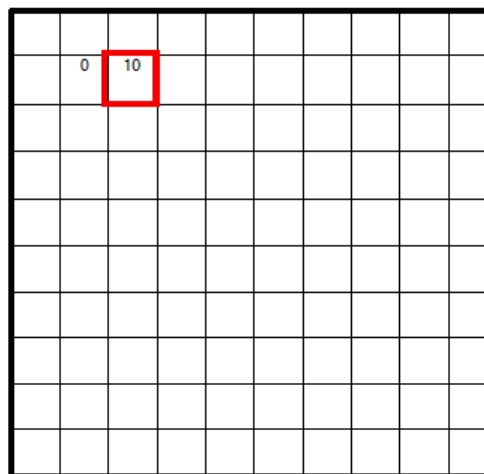


הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$



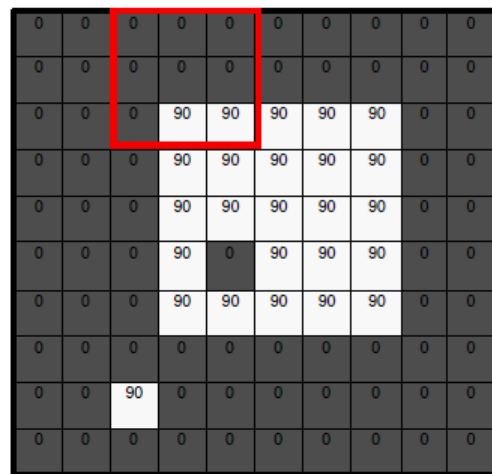
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

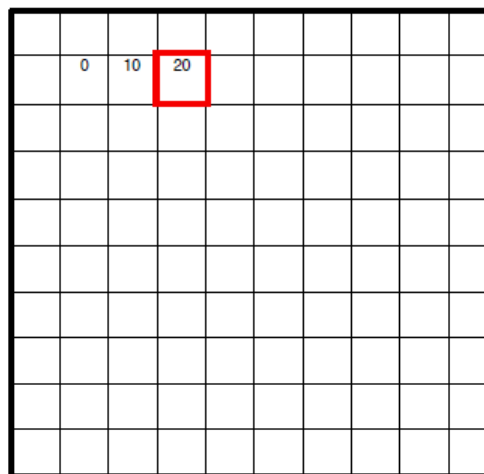


הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$



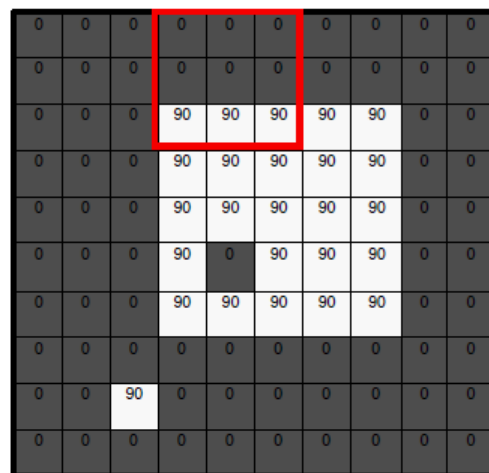
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

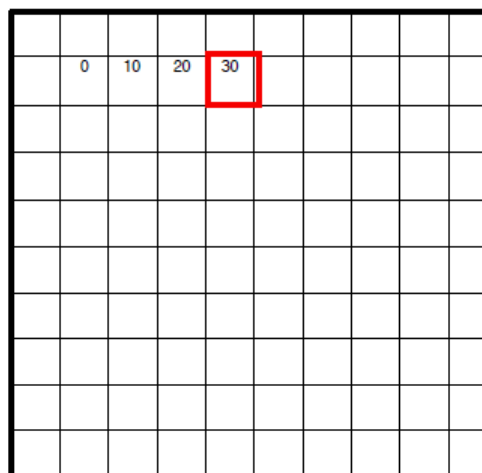


הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$



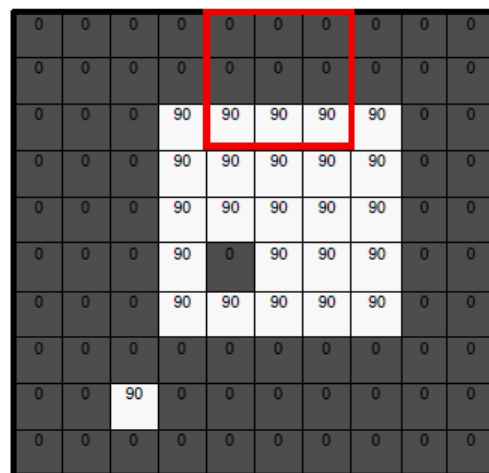
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

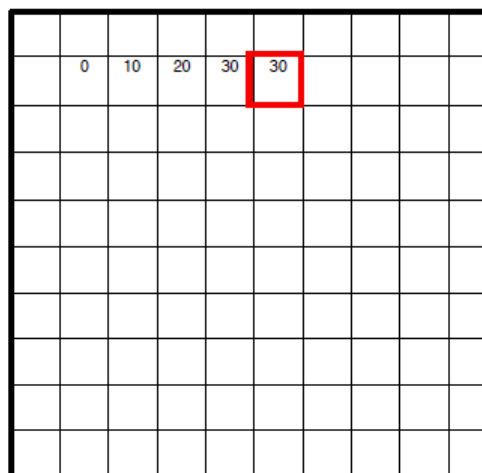


הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$



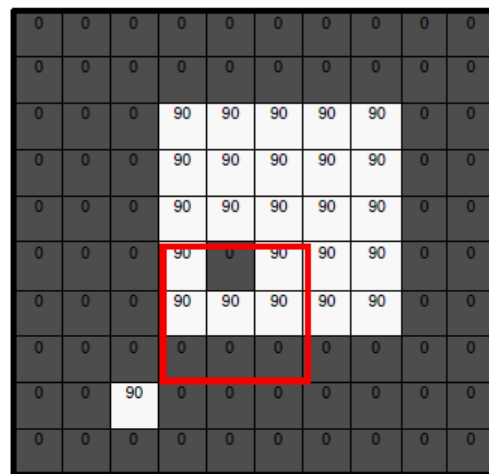
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

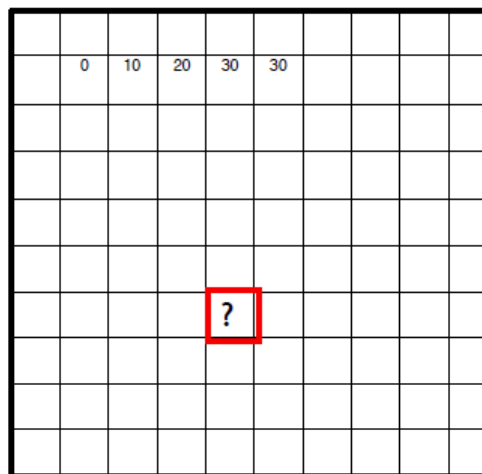


הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$



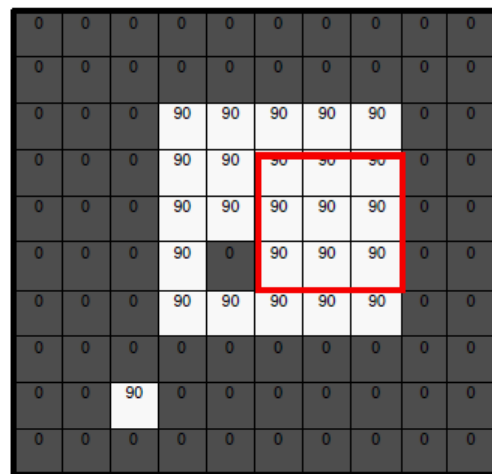
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

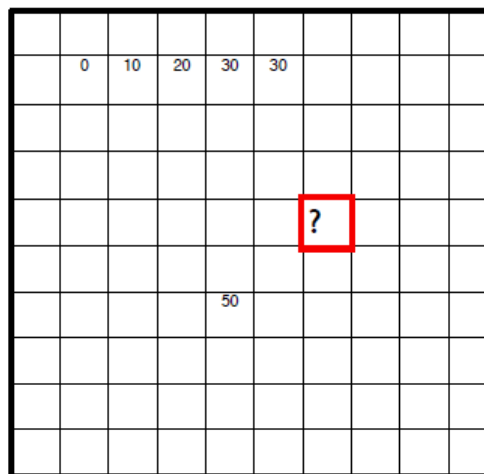


הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$



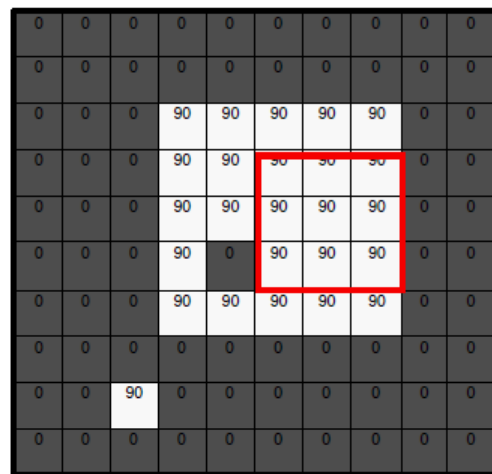
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

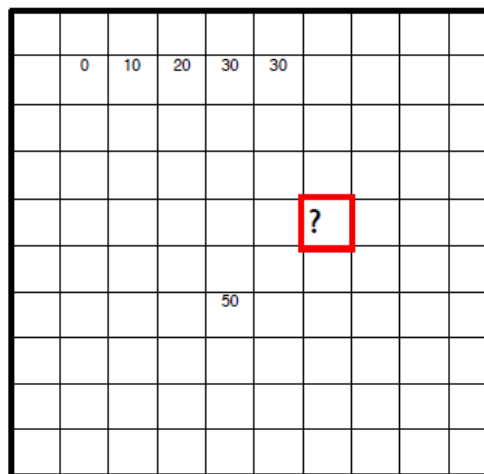


הפילטר

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$



$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

Image filtering

התמונה

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

הפילטר

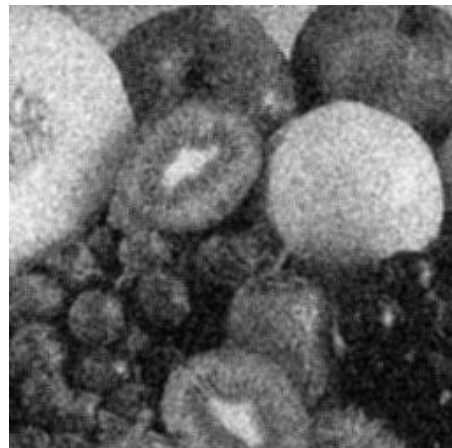
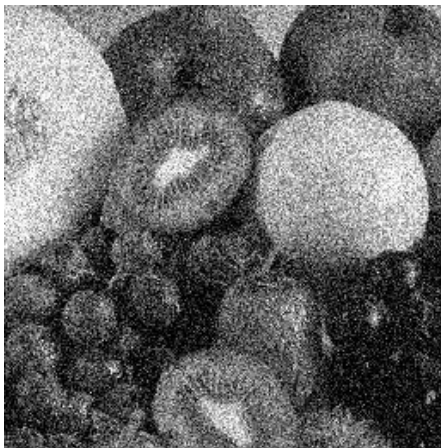
1	1	1
1	1	1
1	1	1

$h[.,.]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

תוצאת
הקונבולוציה
עם הפילטר

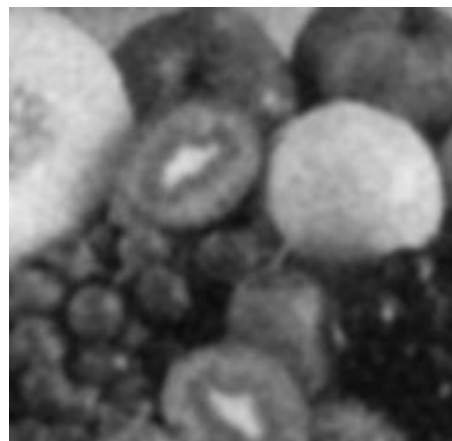


מיצוץ
3X3

1	1	1
1	1	1
1	1	1

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

מיצוץ
5X5



מיצוץ
9X9

1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1

קוד: מיצוע Blur/

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

img = cv2.imread('opencv_logo.png')

kernel = np.ones((5,5),np.float32)/25
dst = cv2.filter2D(img,-1,kernel)

plt.subplot(121),plt.imshow(img),plt.title('Original')
plt.xticks([], plt.yticks([]))
plt.subplot(122),plt.imshow(dst),plt.title('Averaging')
plt.xticks([], plt.yticks([]))
plt.show()
```

קוד: מיצוע / Blur

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

img = cv2.imread('opencv_logo.png')

blur = cv2.blur(img,(5,5))

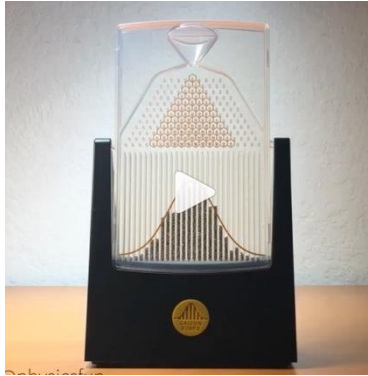
plt.subplot(121),plt.imshow(img),plt.title('Orig')
plt.xticks([], plt.yticks([]))
plt.subplot(122),plt.imshow(blur),plt.title('Blu')
plt.xticks([], plt.yticks([]))
plt.show()
```

עיבוד תמונה – שיעור 3

- Blurring

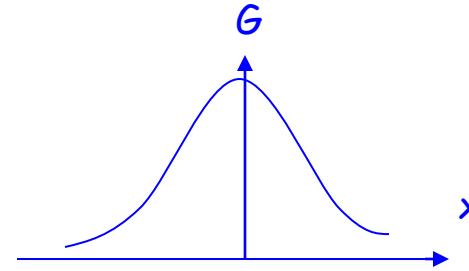
- Edges
- Sharpening
- Features
- Derivatives
- Super-resolution
- Classification
- Detection
- Image captioning
- ...

- .1 חזרה
- .2 Convolutions \ קונבולציה
 - .1 הגדרה
 - .2 דוגמאות ראשונות
 - .3 מה עושים עם קונבולציה – חלק א'
 - .1 Blur
 - .1 אחיד
 - .2 גאוסיאני
 - .3 Bi-lateral filter
 - .2 Edges
 - .1 First derivative

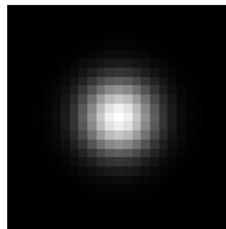
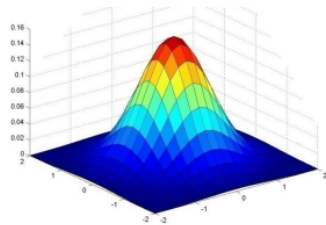


<https://www.facebook.com/physicshunbyrhall/videos/619573898/406561>

גאוס



Gaussian Kernel



$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

$$G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

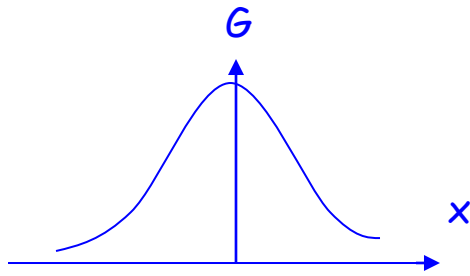
$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

gauss_demo.mp4

מיצוע גאוסיאני



<https://www.facebook.com/physicshunbyrhall/videos/619573898/406561>



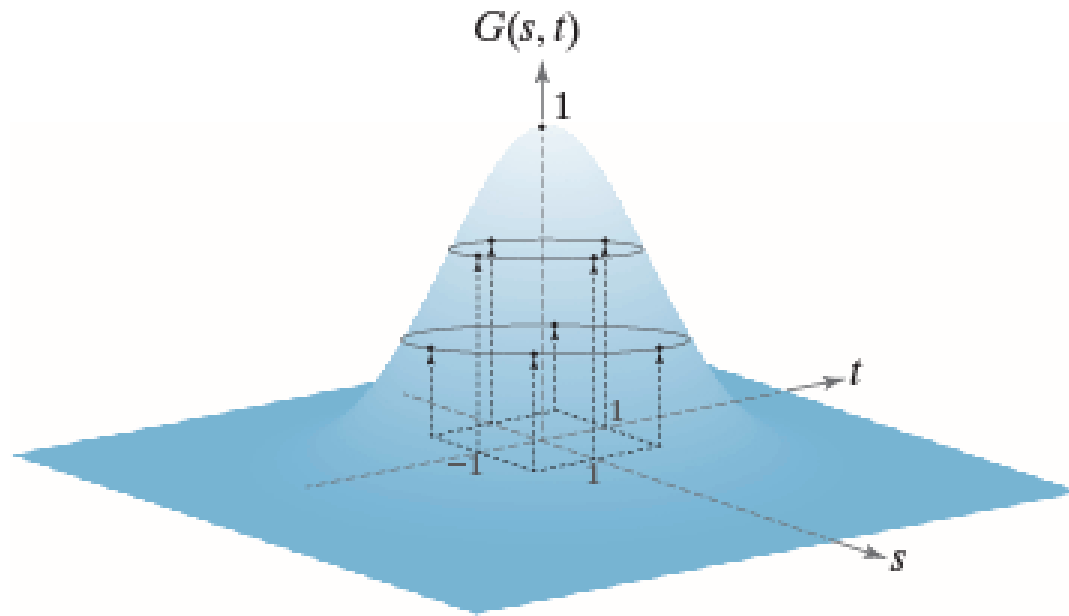
מיצוע עם עדיפות לפיקסל הנוכחי

$$G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

איך נקבל פילטר גאוסייאני ?

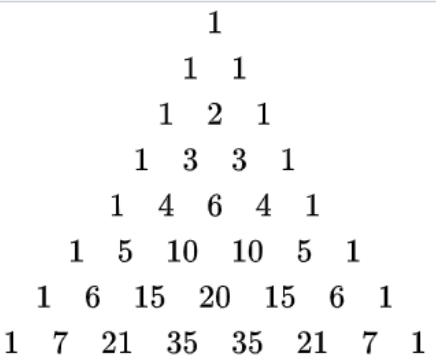
פילטר גאוסיאני \ גאוסיאן \ קרנל גאוסיאני \ מיצוע גאוסיאני.....



$$\frac{1}{4.8976} \times$$

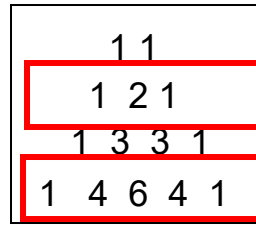
0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679

שיטה 1 – לקבלת פילטר גאוסיאני

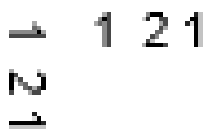


The binomial coefficient $\binom{n}{b}$ appears as the b th entry in the n th row of Pascal's triangle (counting starts at 0). Each entry is the sum of the two above it.

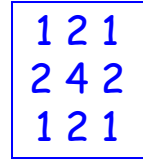
We usually use the binomial coefficients instead:



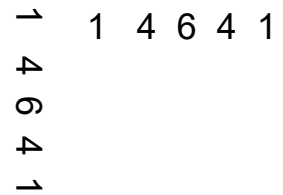
שיטה זו – לקבלת פילטר גאוסייאני



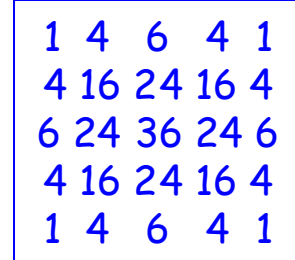
$(1/16)^*$



פילטר
גאוסייאני
3x3



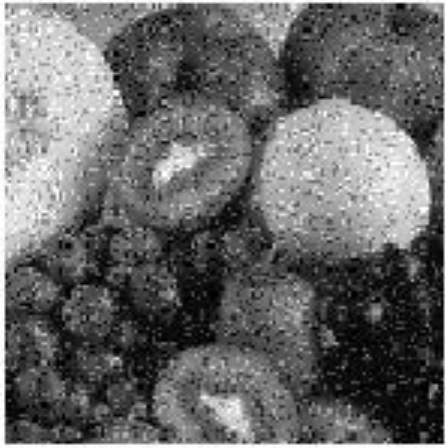
$(1/256)^*$



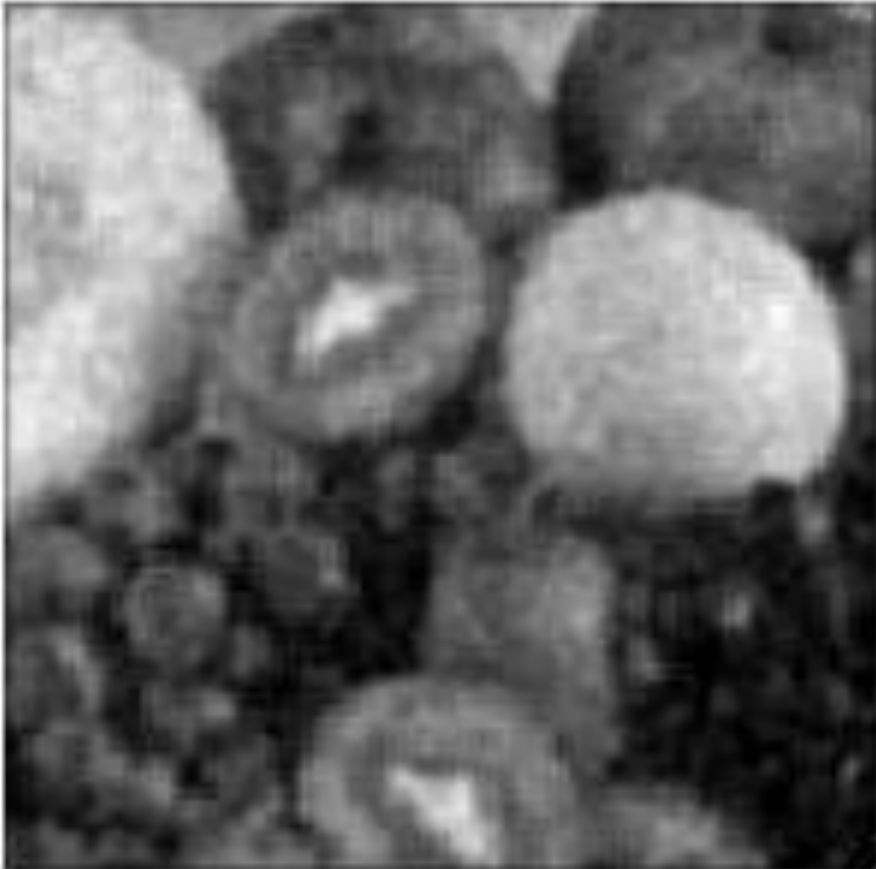
פילטר
גאוסייאני
5x5

שיטה זו – לקבלת פילטר גאוסייאני (או, בוא נדבר תכלס...)

```
blur = cv2.GaussianBlur(img,(5,5),0)
```



אחרי
מיצוע
אחיד
3X3

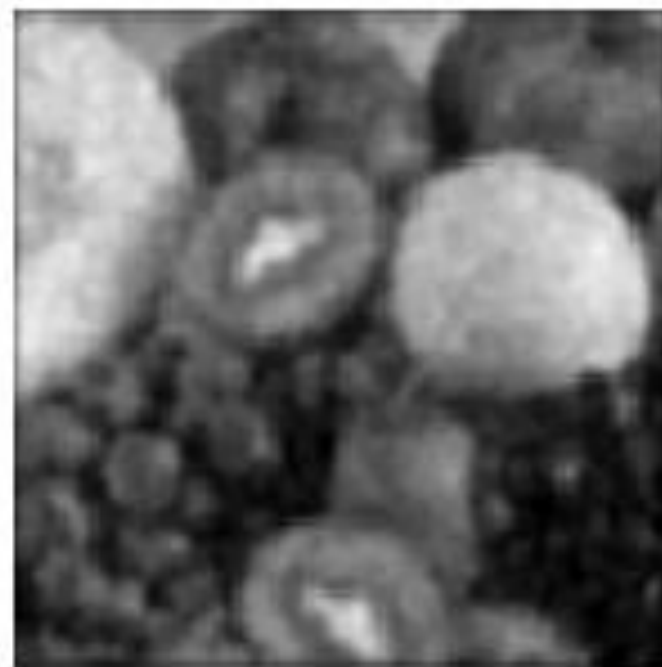
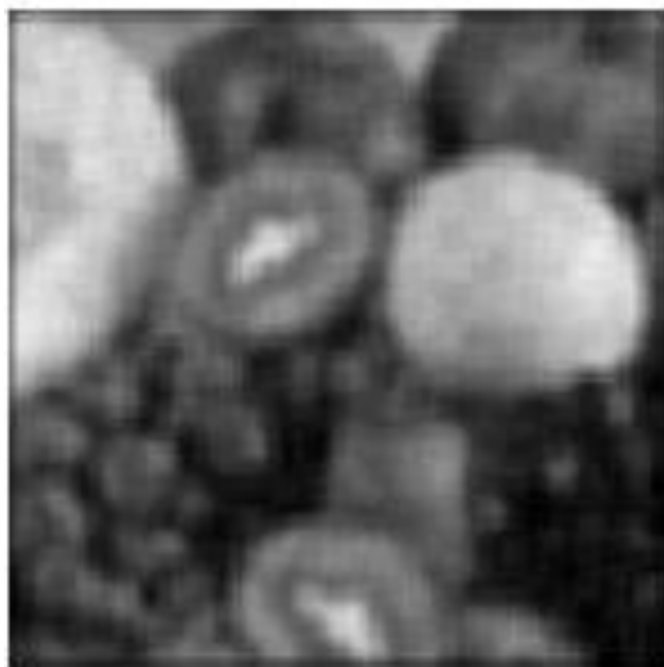


אחרי
מיצוע
גאוס'אני
3X3

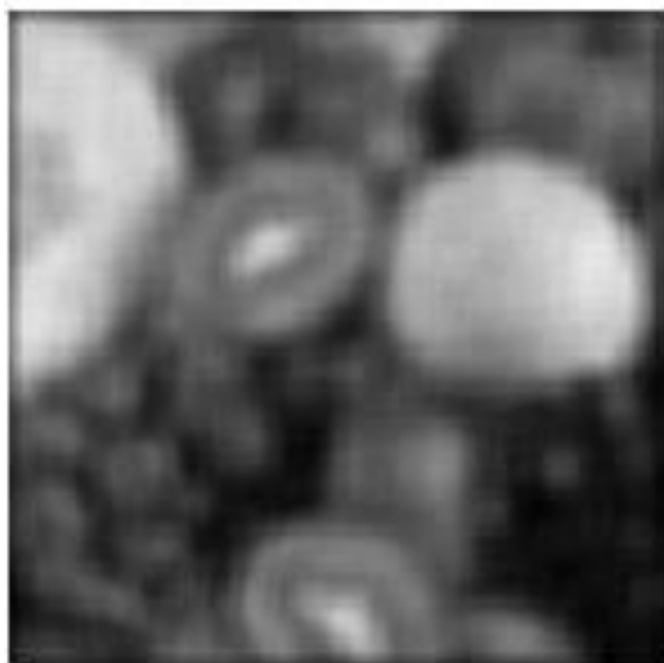




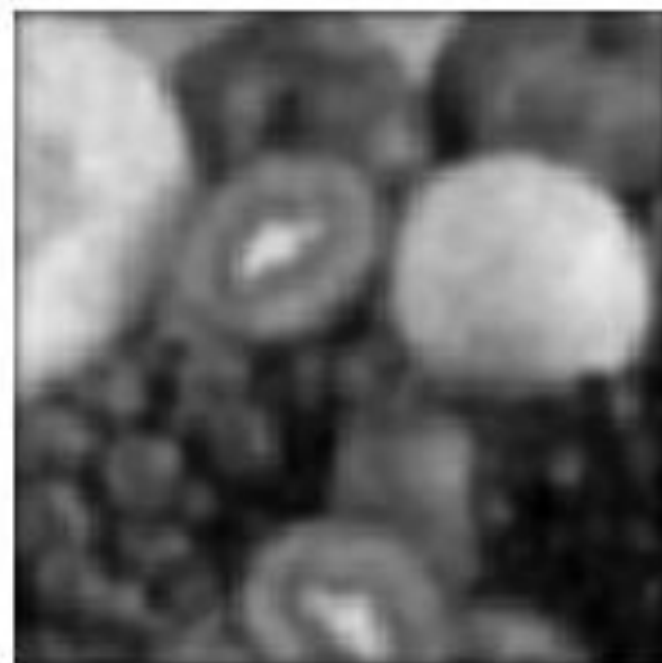
מיצוץ
5X5



מיצוץ
גאוס'אני
5X5



מיצוץ
7X7



מיצוץ
גאוס'אני
7X7

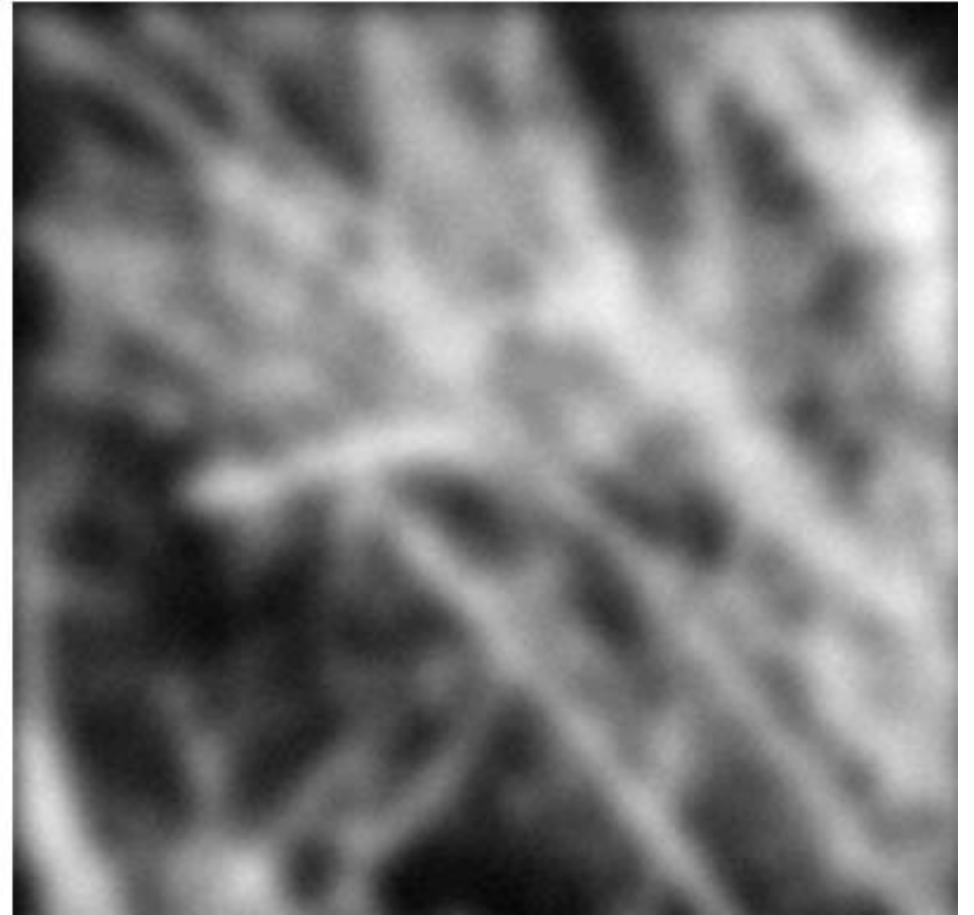
Smoothing with Gaussian filter

הפילטר



לפני

אחרי



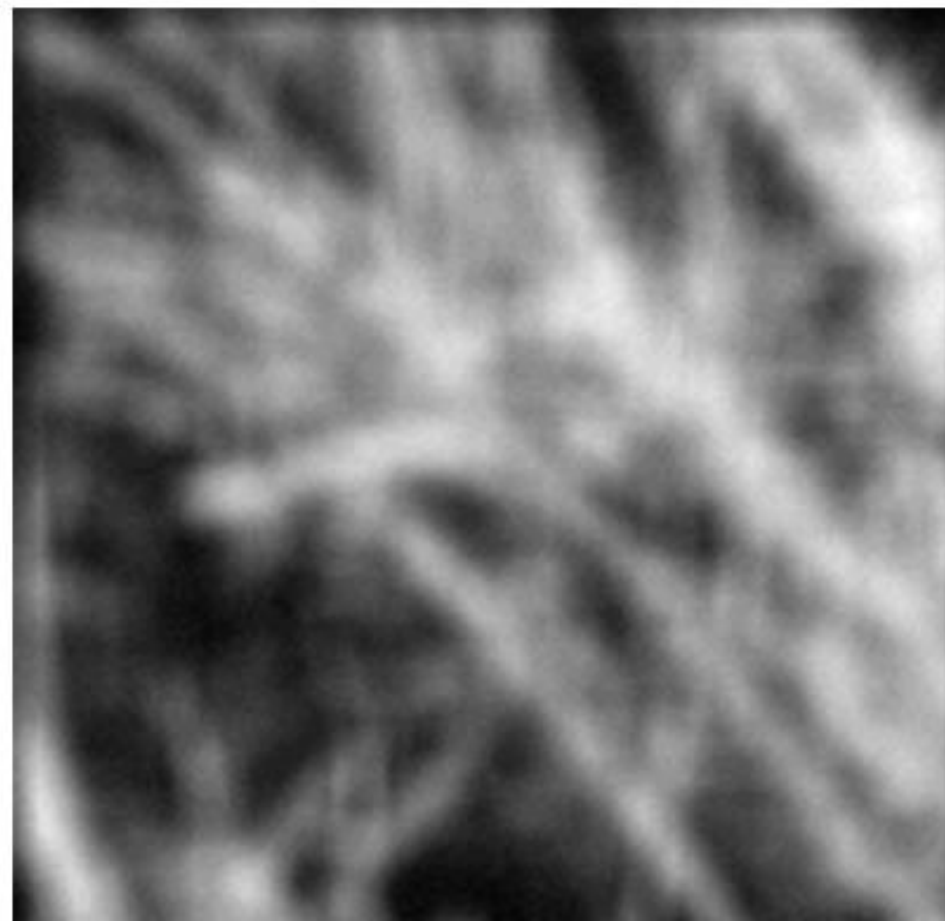
Smoothing with box filter

הפילטר

לפני



אחרי



Smoothing with box filter

עוד מילים
נרדפות ל
"box filter"
Uniform

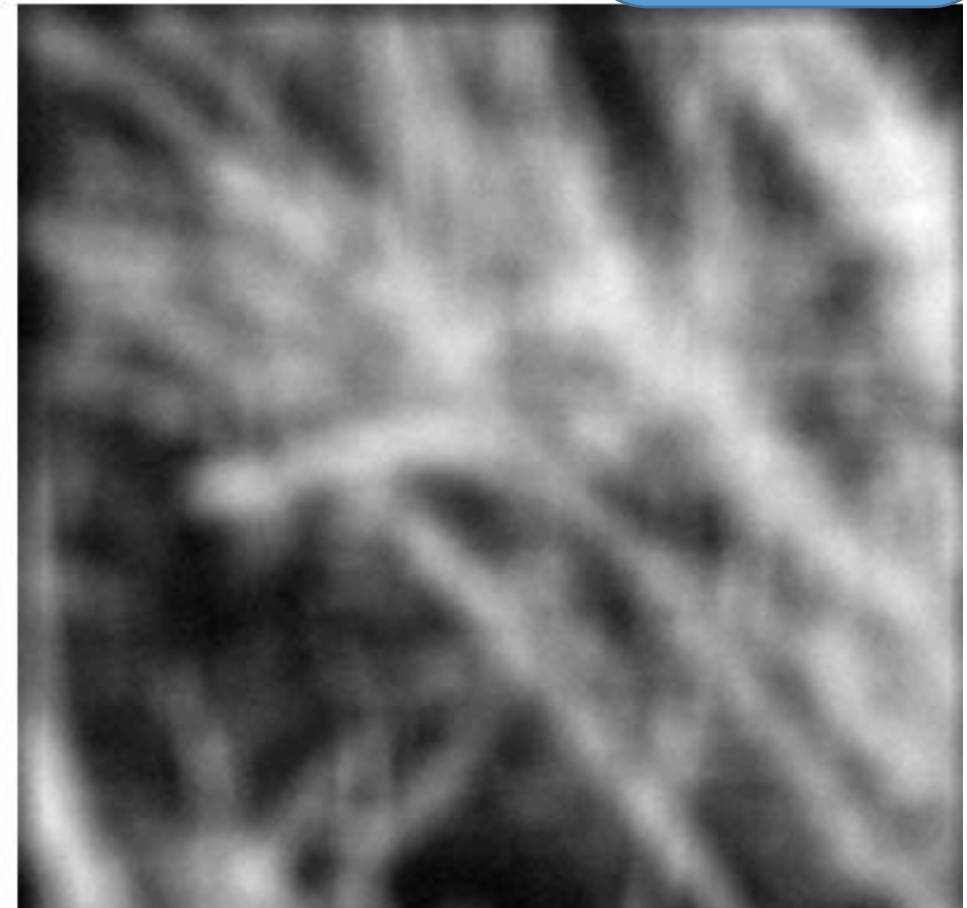
הפילטר



אחרי

לפני

עוד מילים
נרדפות ל
"smoothing"
Blur
החלקה
מיצוע



"מילים ורדפות"

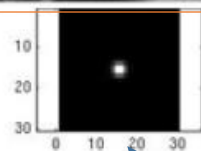
Smoothing with box filter	Smoothing with Gaussian filter
Smoothing with uniform filter	Smoothing with Gaussian
Blur with box filter	Blur with Gaussian
Blur with uniform filter	Blur with Gaussian filter
Averaging with uniform filter	Averaging with Gaussian filter
Blur with box filter	Blur with Gaussian filter
box filter	Smoothing.
Smoothing.	Averaging
Averaging	Blurring.
Blurring	Gaussian Blurring
homogeneous	

"מילים ורדפות"

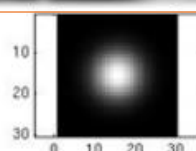
Smoothing with box filter	Smoothing with Gaussian filter
Smoothing with uniform filter	Smoothing with Gaussian
Blur with box filter	Blur with Gaussian
Blur with uniform filter	Blur with Gaussian filter
Averaging with uniform filter	Averaging with Gaussian filter
Blur with box filter	Blur with Gaussian filter
box filter	Smoothing.
Smoothing.	Averaging
Averaging	Blurring.
Blurring	Gaussian Blurring
homogeneous	

עוד דוגמא

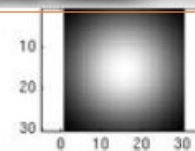
Parameter σ is the “scale” / “width” / “spread” of the Gaussian kernel, and controls the amount of smoothing.



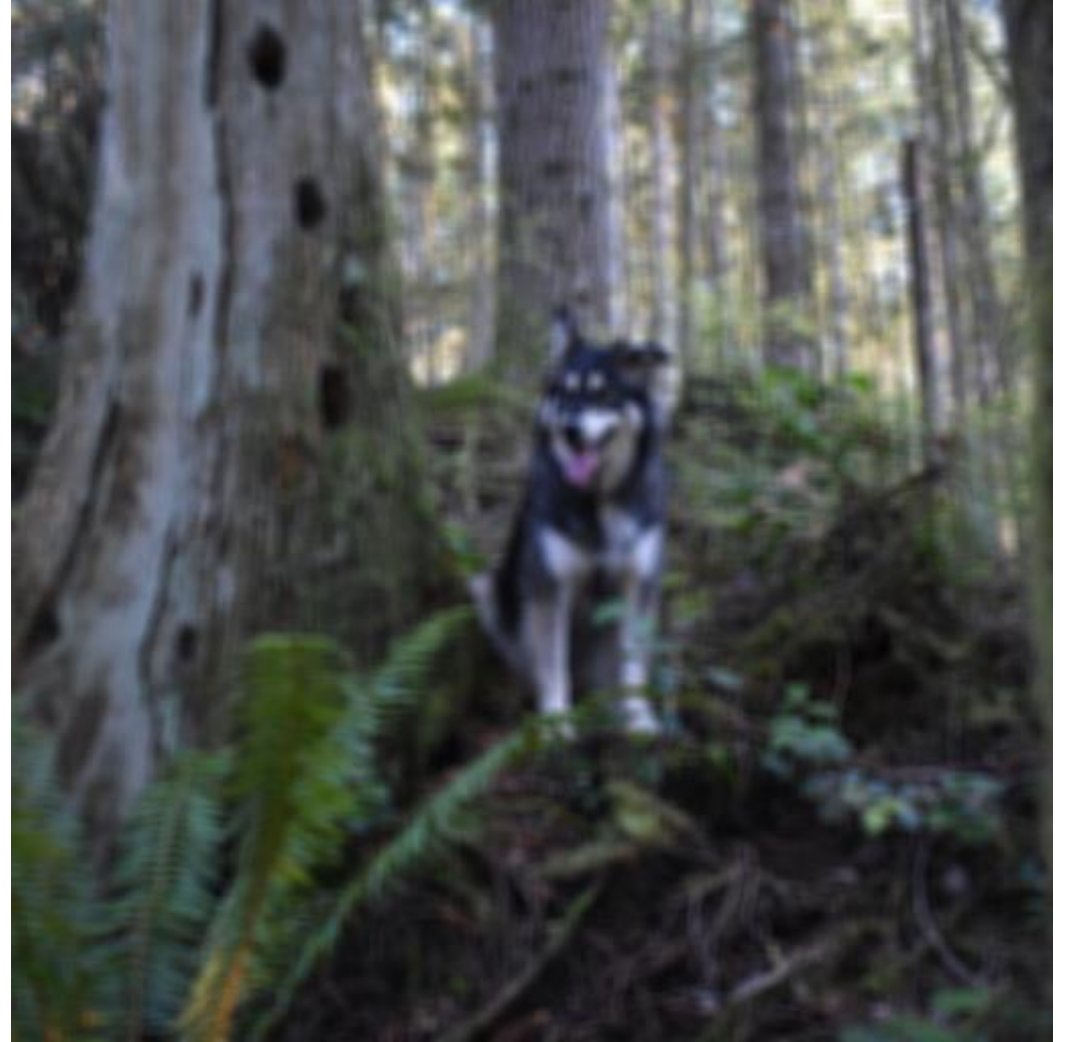
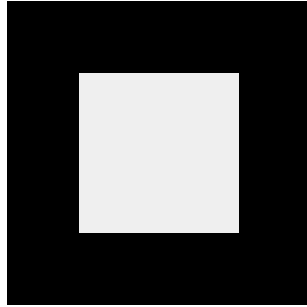
הפילטר



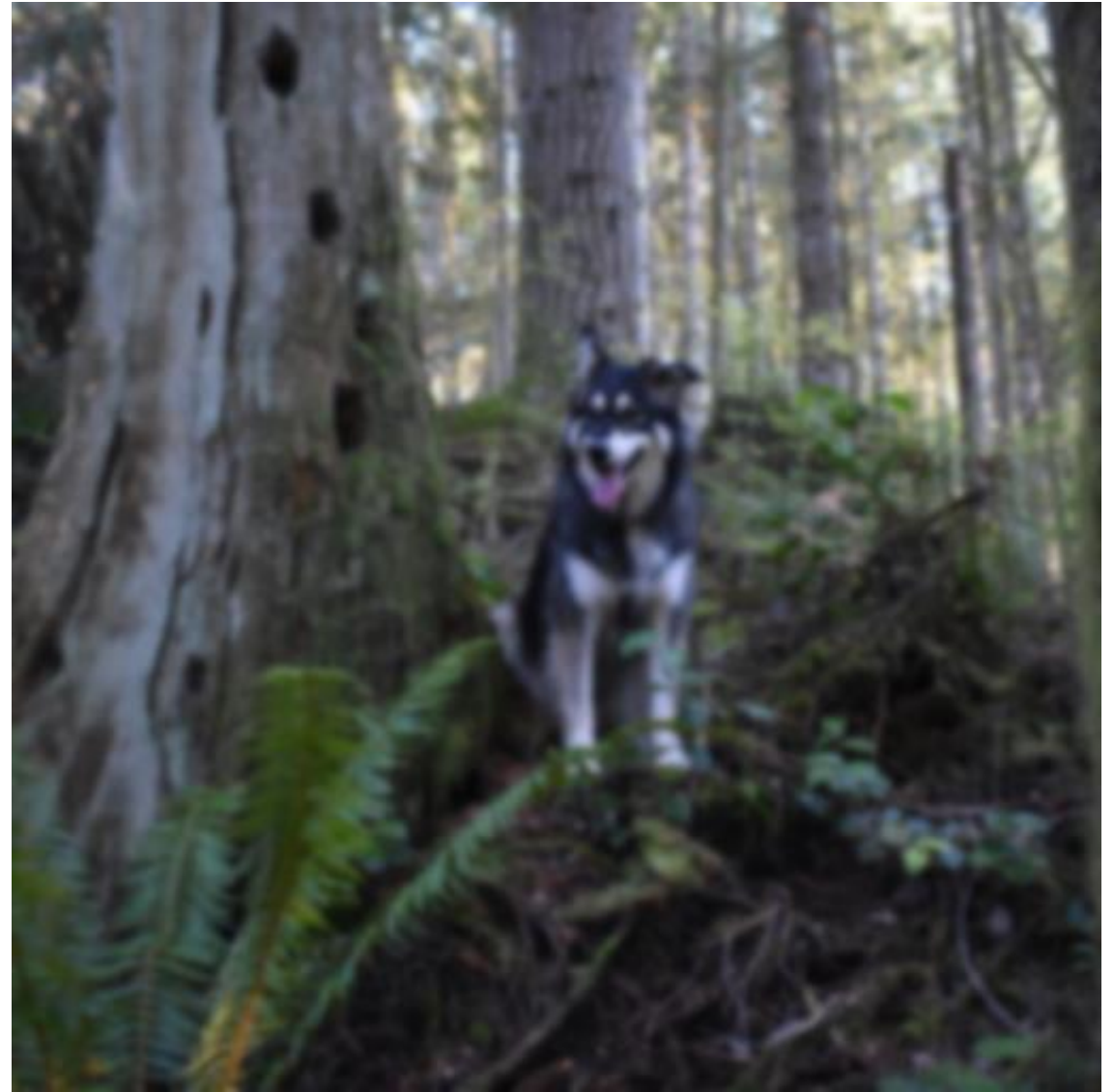
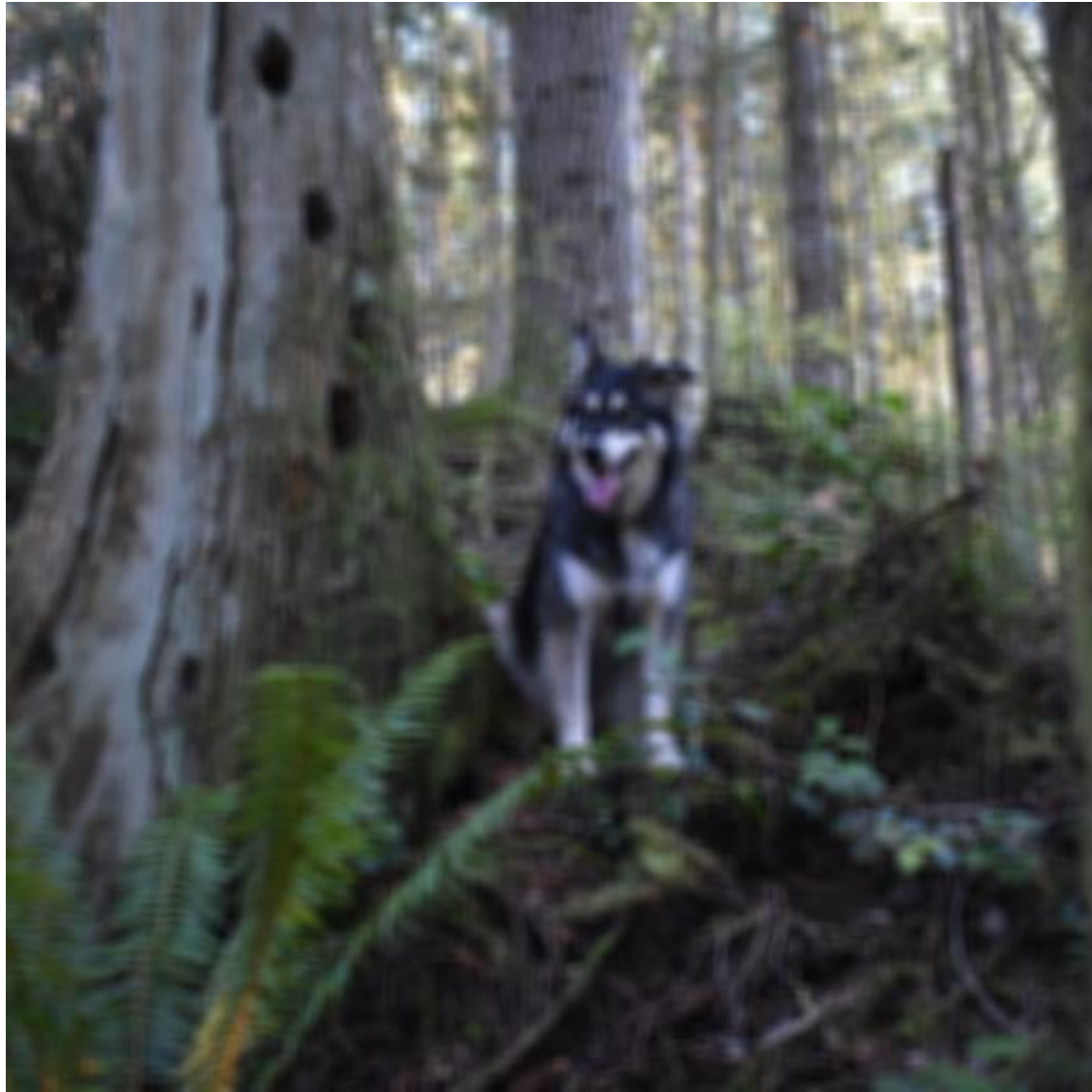
...



We want a smoothly weighted kernel



Better smoothing with Gaussians



Better smoothing with Gaussians



עיבוד תמונה – שיעור 3

- Blurring

- Edges
- Sharpening
- Features
- Derivatives
- Super-resolution
- Classification
- Detection
- Image captioning
- ...

- .1 חזרה
- .2 Convolutions \ קונבולציה
 - .1 הגדרה
 - .2 דוגמאות ראשונות
 - .3 מה עושים עם קונבולציה – חלק א'
 - .1 Blur
 - .1 אחיד
 - .2 גאוסיאני
 - .3 Bi-lateral filter
 - .2 Edges
 - .1 First derivative

**“Fixing the Gaussian
Blur”:**

the Bilateral Filter

input



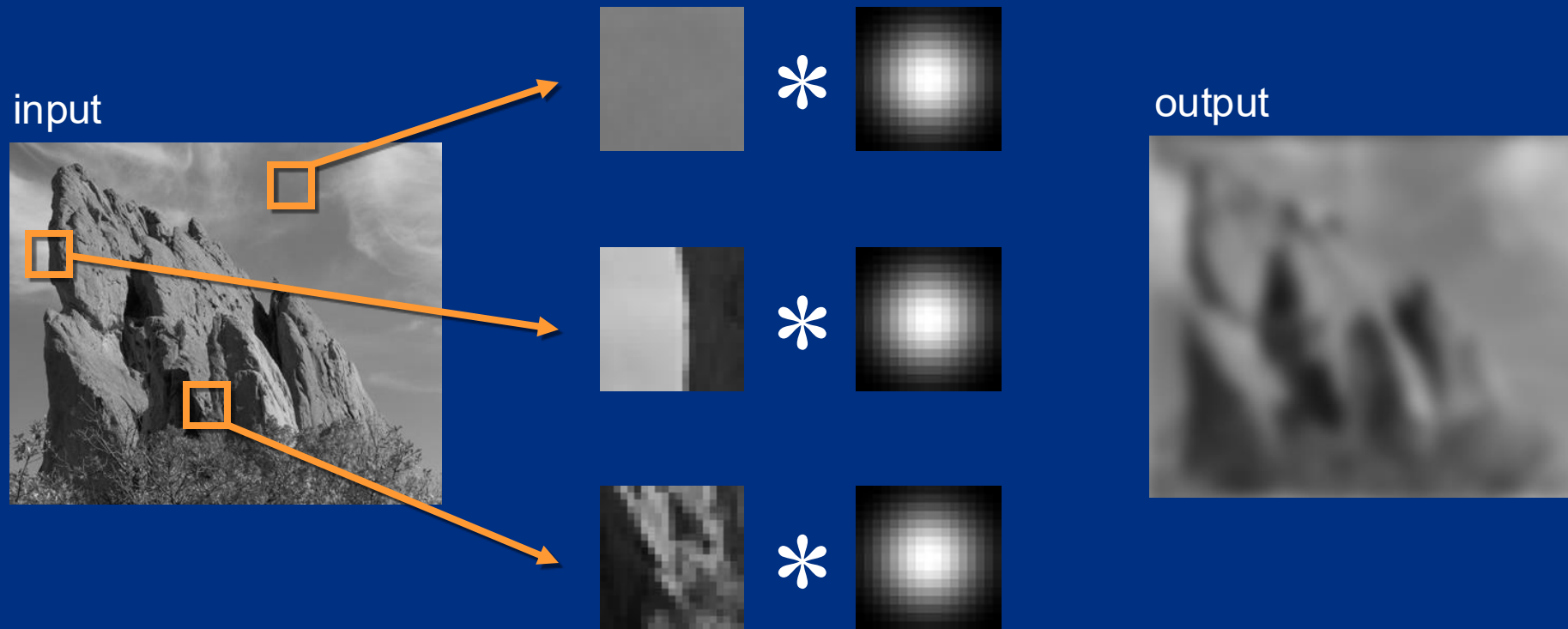
box average

Gaussian blur



The Bilateral Filter

Blur Comes from Averaging across Edges

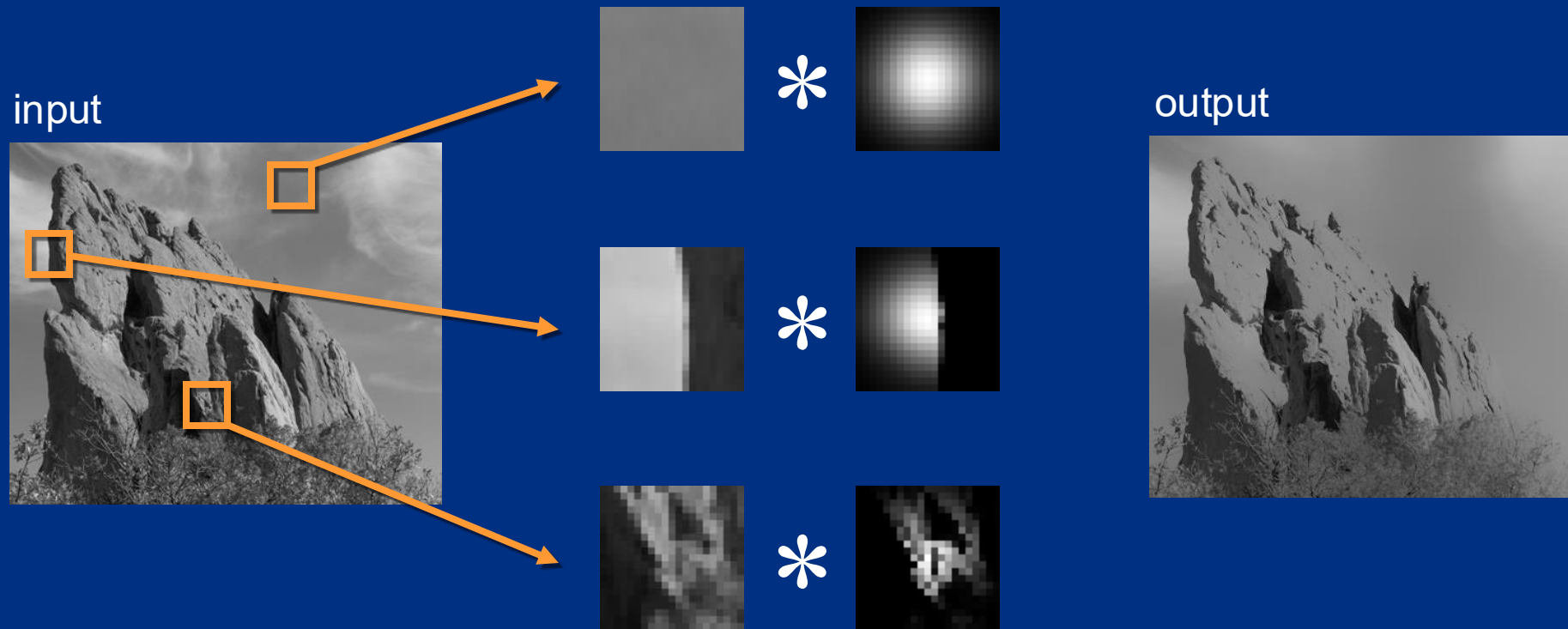


Same Gaussian kernel everywhere.

Bilateral Filter

No Averaging across Edges

[Aurich 95, Smith 97, Tomasi 98]



The kernel shape depends on the image content.

Bilateral Filter Definition: an Additional Edge Term

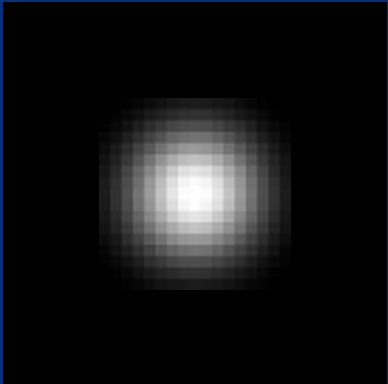
Same idea: **weighted average of pixels.**

$$BF[I]_p = \overset{\text{new}}{\frac{1}{W_p}} \sum_{q \in S} \overset{\text{not new}}{G_{\sigma_s}(\|p - q\|)} \overset{\text{new}}{G_{\sigma_r}(\|I_p - I_q\|)} I_q$$

normalization factor

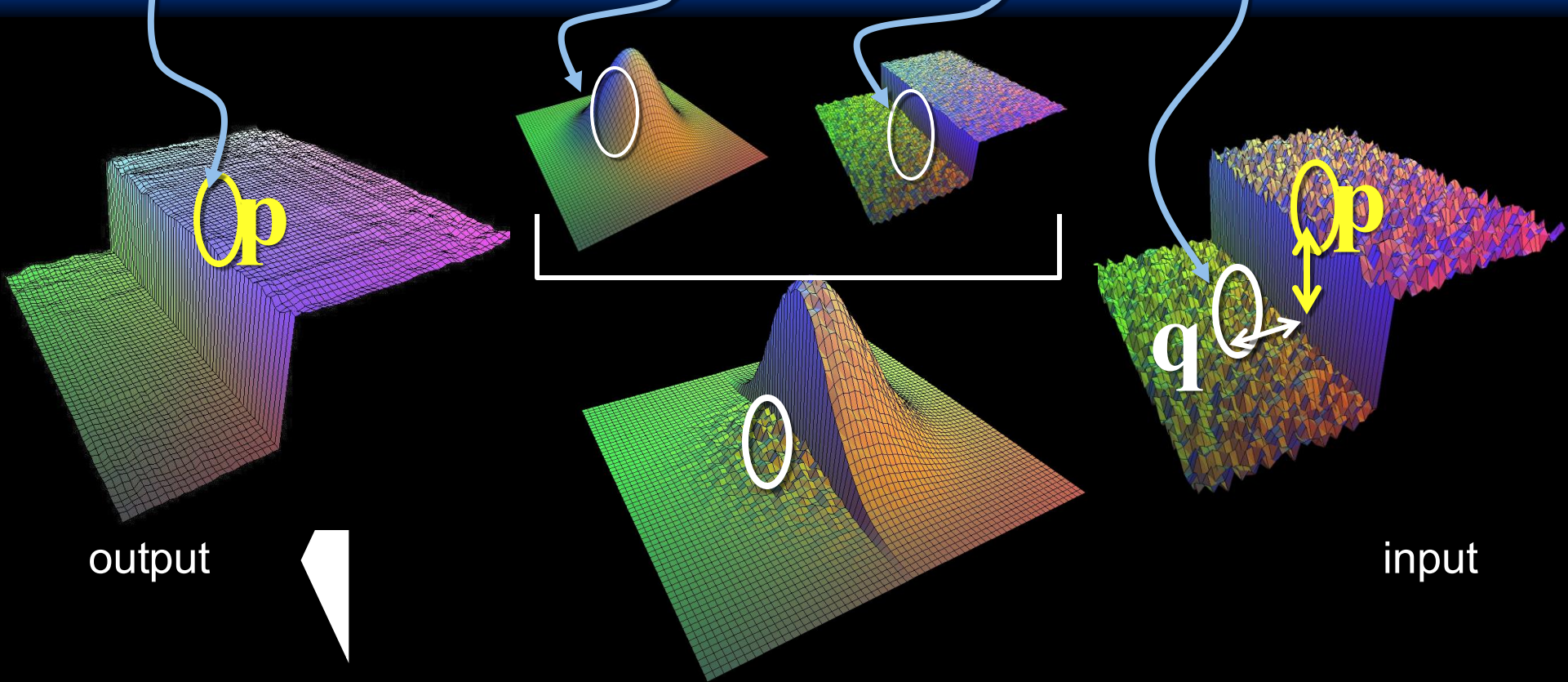
space weight

range weight



Bilateral Filter on a Height Field

$$\underbrace{BF[I]_p}_{\text{output}} = \frac{1}{W_p} \sum_{\mathbf{q} \in S} \underbrace{G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|)}_{\text{spatial}} \underbrace{G_{\sigma_r}(|I_p - I_q|)}_{\text{range}} \underbrace{I_q}_{\text{input}}$$



Space and Range Parameters

$$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in S} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|) G_{\sigma_r}(|I_{\mathbf{p}} - I_{\mathbf{q}}|) I_{\mathbf{q}}$$


- space σ_s : spatial extent of the kernel, size of the considered neighborhood.
- range σ_r : “minimum” amplitude of an edge

openCV

Python: `cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace[, dst[, borderType]])` → `dst`

- Parameters:**
- **src** – Source 8-bit or floating-point, 1-channel or 3-channel image.
 - **dst** – Destination image of the same size and type as `src`.
 - **d** – Diameter of each pixel neighborhood that is used during filtering. If it is non-positive, it is computed from `sigmaSpace`.
 - **sigmaColor** – Filter sigma in the color space. A larger value of the parameter means that farther colors within the pixel neighborhood (see `sigmaSpace`) will be mixed together, resulting in larger areas of semi-equal color.
 - **sigmaSpace** – Filter sigma in the coordinate space. A larger value of the parameter means that farther pixels will influence each other as long as their colors are close enough (see `sigmaColor`). When $d > 0$, it specifies the neighborhood size regardless of `sigmaSpace`. Otherwise, `d` is proportional to `sigmaSpace`.

new_lecture_code\lect03_bf



```
for i in range(1, MAX_KERNEL_LENGTH, 2):  
    dst = cv.blur(src, (i, i))  
    if display_dst(DELAY_BLUR, capt, i) != 0:  
        return 0
```

```
for i in range(1, MAX_KERNEL_LENGTH, 2):  
    dst = cv.GaussianBlur(src, (i, i), 0)  
    if display_dst(DELAY_BLUR, capt, i) != 0:  
        return 0
```

```
for i in range(1, MAX_KERNEL_LENGTH, 2):  
    dst = cv.medianBlur(src, i)  
    if display_dst(DELAY_BLUR, capt, i) != 0:  
        return 0
```

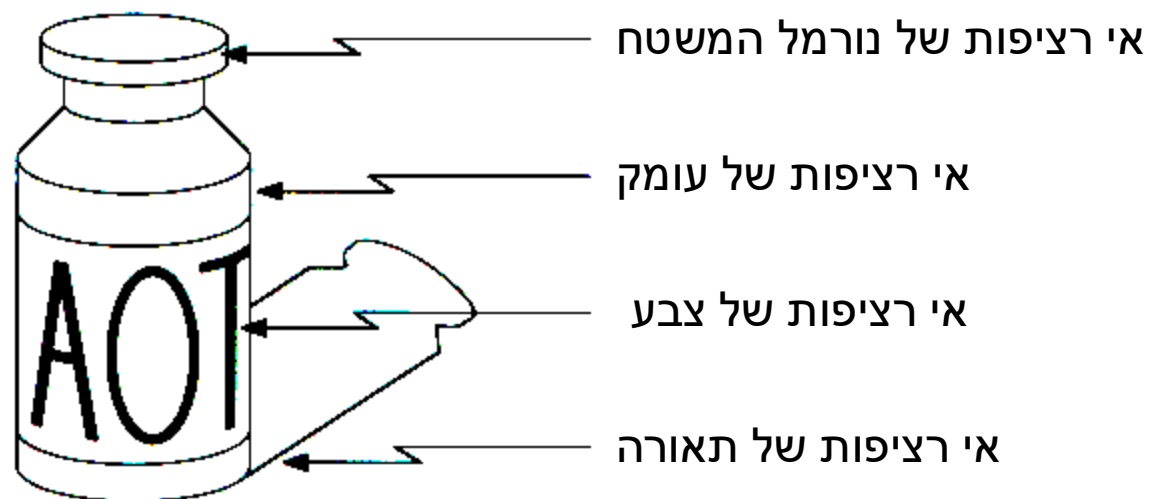
```
for i in range(1, MAX_KERNEL_LENGTH, 2):  
    dst = cv.bilateralFilter(src, i, i * 2, i / 2)  
    if display_dst(DELAY_BLUR, capt, i) != 0:  
        return 0
```

עיבוד תמונה – שיעור 3

- Blurring
- Edges
- Sharpening
- Features
- Derivatives
- Super-resolution
- Classification
- Detection
- Image captioning
- ...

- .1 חזרה
- .2 Convolutions \ קונבולציה
- .1 הגדרה
- .2 דוגמאות ראשונות
- .3 מה עושים עם קונבולציה – חלק א'
- .1 Blur
- .1 אחיד
- .2 גאוסיאני
- .3 Bi-lateral filter
- .2 Edges
- .1 First derivative

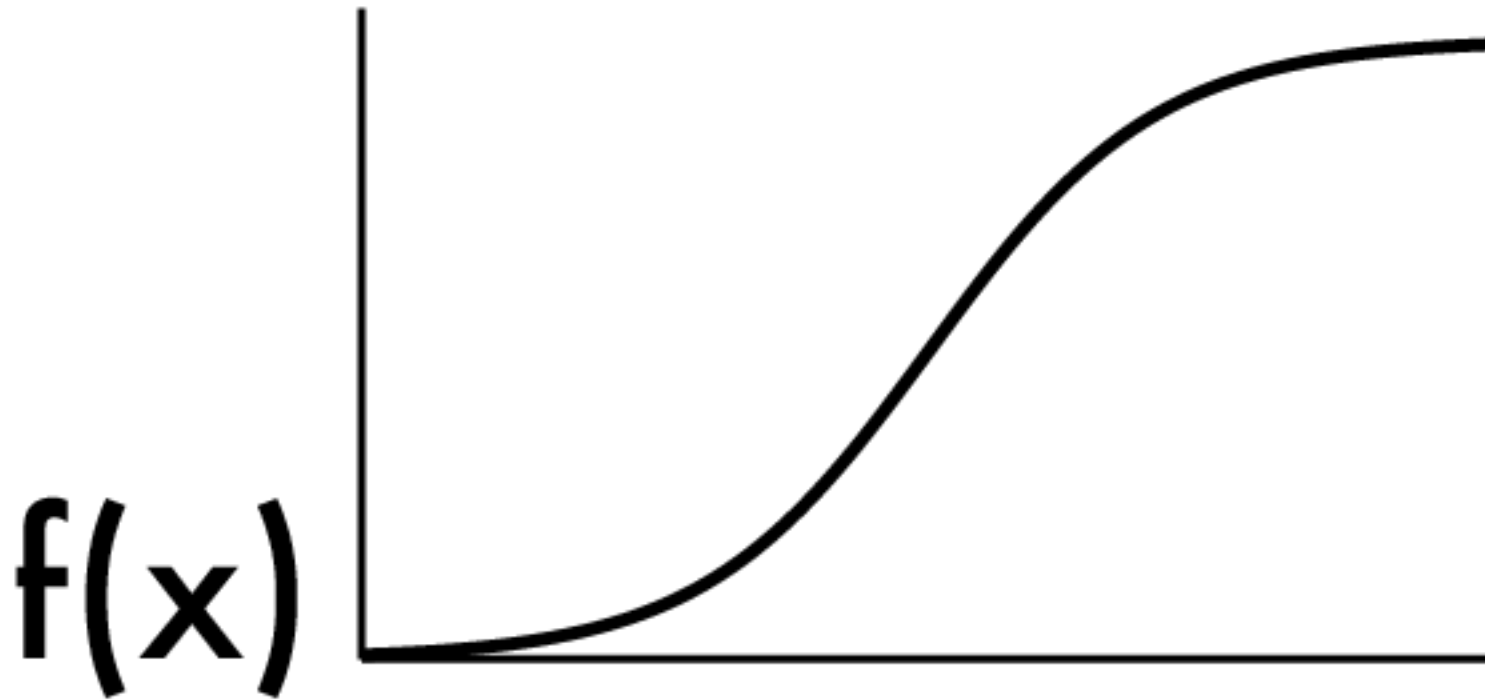
מקורות אפשריים לקצוות

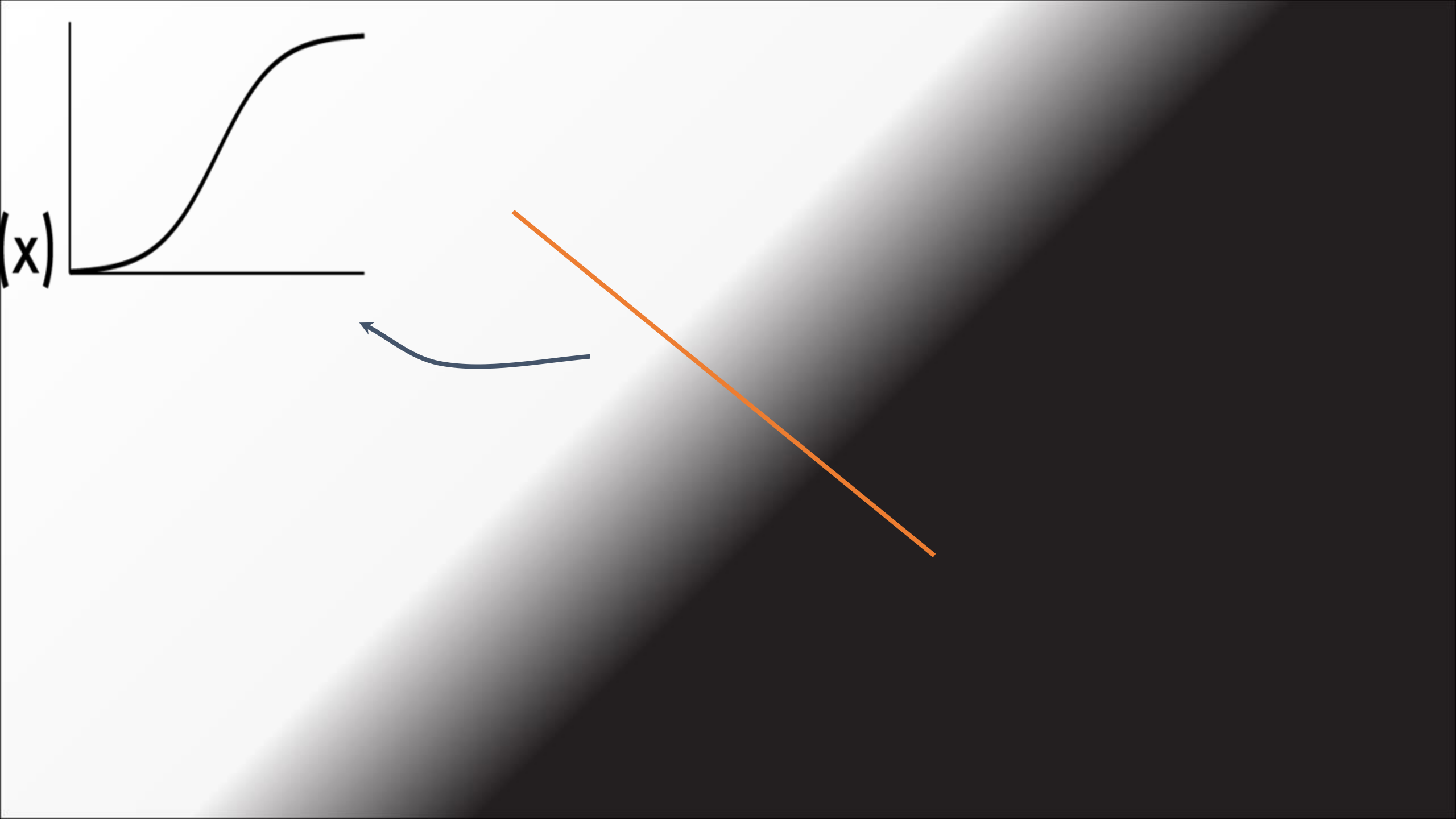


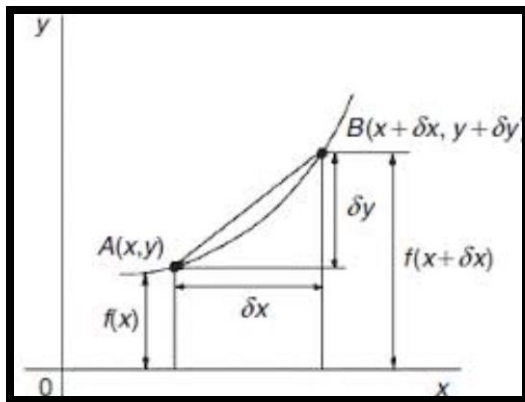
[Cornelia Fermüller](#)

What's an edge?

- Image is a function
- Edges are rapid changes in this function





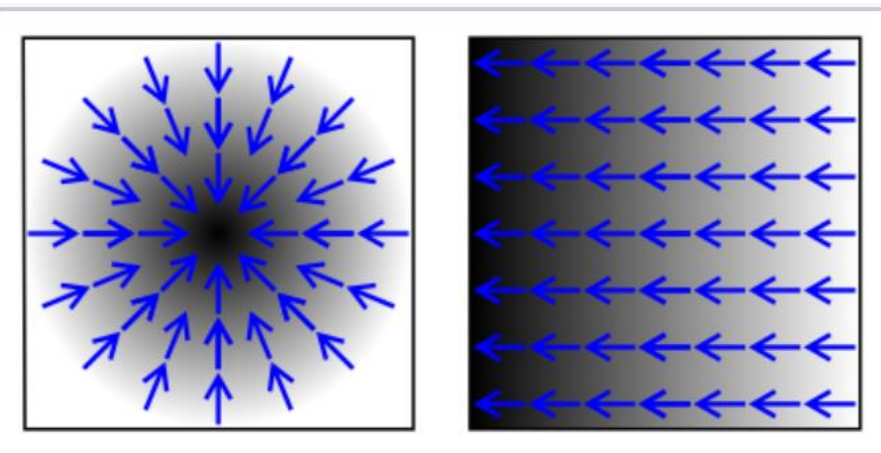


הגדרה – גרדיאנט – "תזכורת"

גרדיאנט

גרדיאנט הוא הכללה של מושג הנגזרת בעבור חשבון אינפיניטסימלי של מספר משתנים. הגרדיאנט הוא אופרטור וקטורי המופעל על שדה סקלרי. הגרדיאנט של שדה סקלרי הוא שדה וקטורי המשייך לכל נקודה במרחב וקטור.

כיוון וקטור הגרדיאנט מצביע אל הכיוון בו השינוי בשדה הסקלרי מקסימלי (חיובי). גודל וקטור הגרדיאנט כשיעור השינוי המקסימלי.



המחשה של גרדיאנט. באיורים האלה, השדה הסקלרי מתואר באמצעות שינוי הצבע, כאשר אזורים כהים יותר הם ערכים גדולים יותר של הפונקציה. החצים הכחולים מתארים את הגרדיאנט הנגזר מהשדה הסקלרי. החצים פונים אל עבר האזורים הגבוהים יותר.

הגדרה – גרדיאנט \ גרדיאנט צבע

גרדיאנט

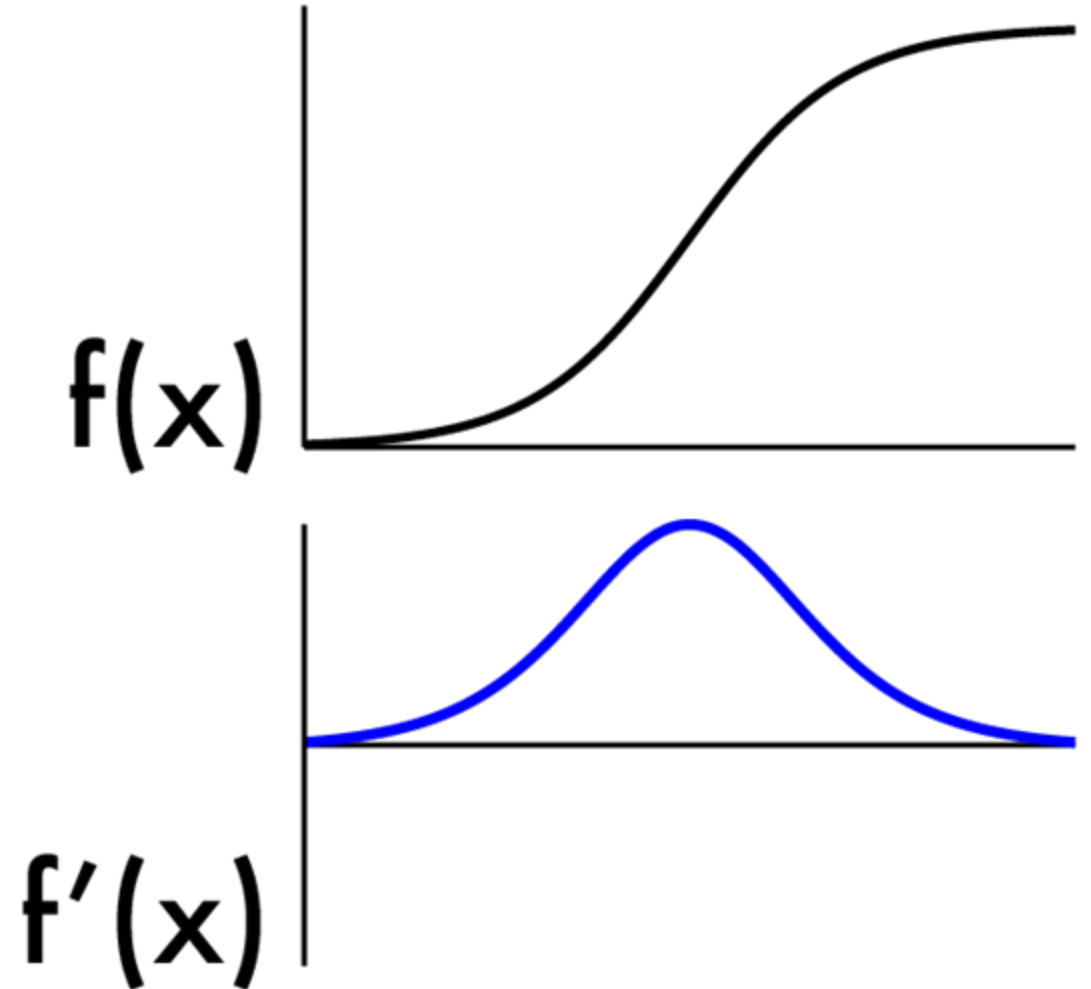
גרדיאנט הוא הכללה של מושג הנגזרת בעבור חשבון אינפיניטסימלי של מספר משתנים. הגרדיאנט הוא אופרטור וקטורי המופעל על שדה סקלרי. הגרדיאנט של שדה סקלרי הוא שדה וקטורי המשייך לכל נקודה במרחב וקטור.

כיוון וקטור הגרדיאנט מצביע אל הכיוון בו השינוי בשדה הסקלרי מקסימלי (חיובי). גודל וקטור הגרדיאנט כשיעור השינוי המקסימלי.



Finding edges

- Could take derivative
- Edges = high response



■ Recall

■ We could approximate this as

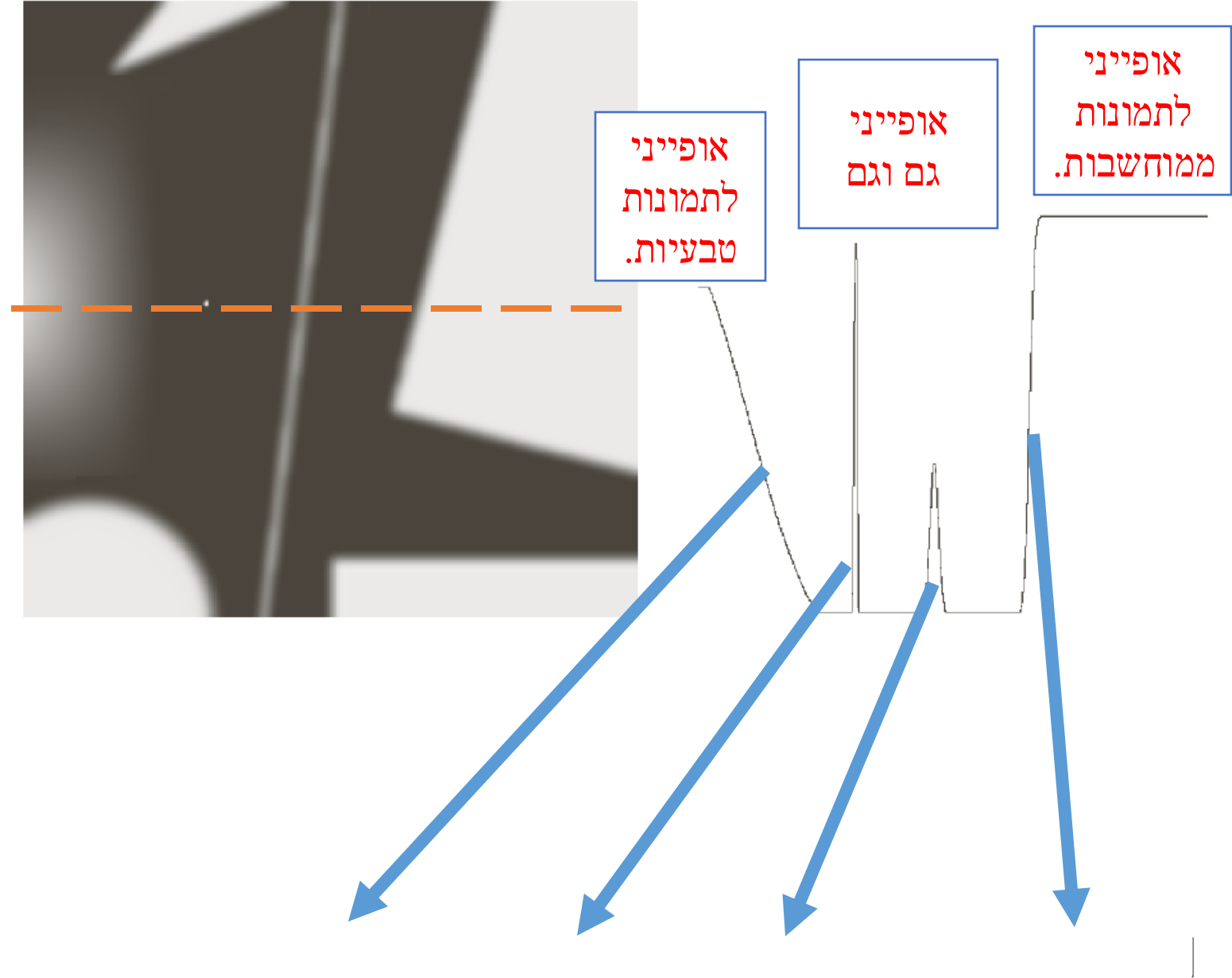
$$\frac{\partial}{\partial x} f = \lim_{\varepsilon \rightarrow 0} \left(\frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon} \right) \quad \frac{\partial}{\partial x} f \approx \frac{f(x_{i+1}, y) - f(x_{i-1}, y)}{2 \cdot \Delta x}$$



$$\frac{\partial f}{\partial x} \approx f(x+1, y) - f(x, y)$$

$$\frac{\partial f}{\partial y} \approx f(x, y+1) - f(x, y)$$

שש



שש

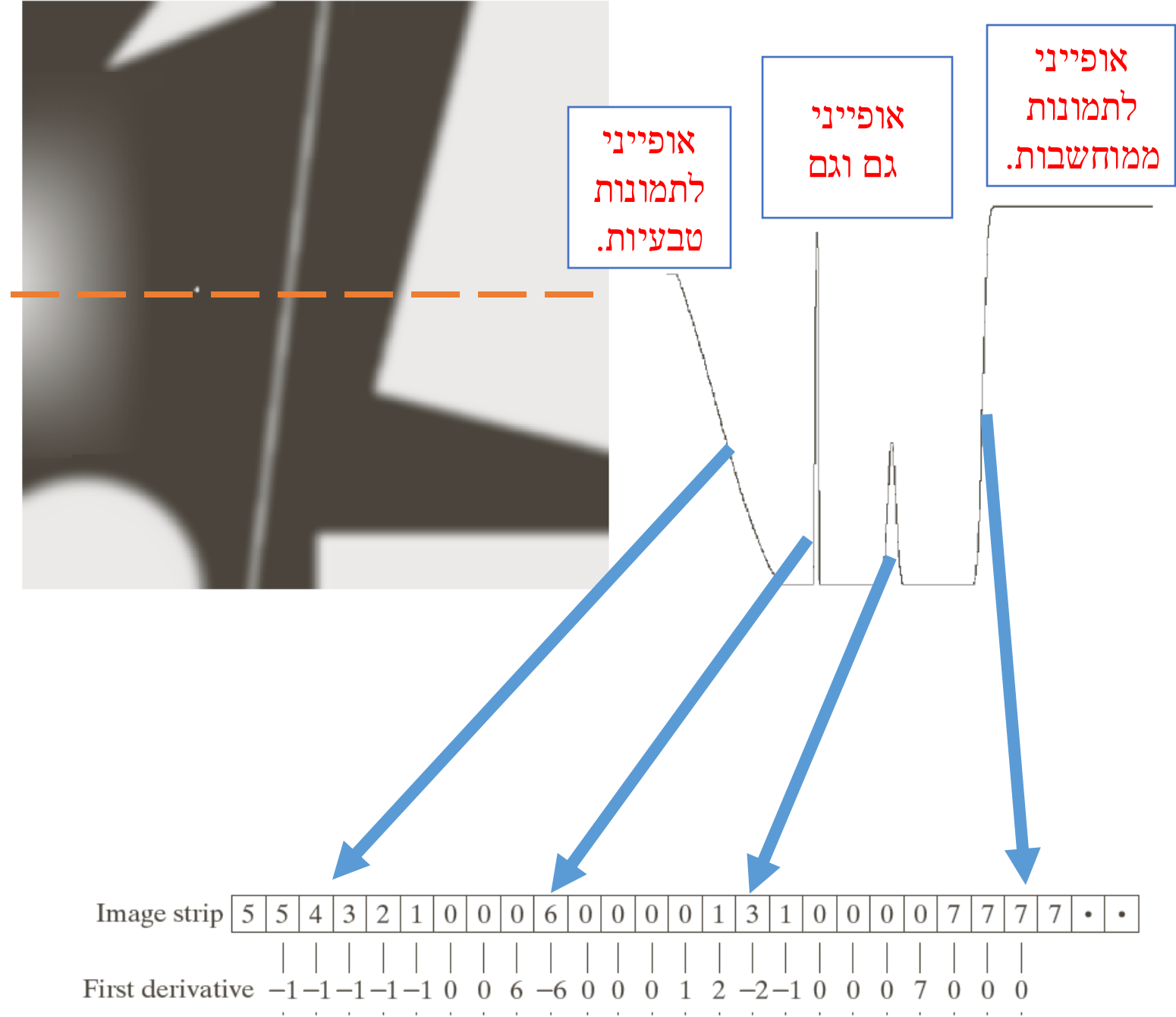


Image derivatives

- Recall:

- $f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$

- We don't have an "actual" Function, must estimate
- Possibility: set $h = 1$
- What will that look like?

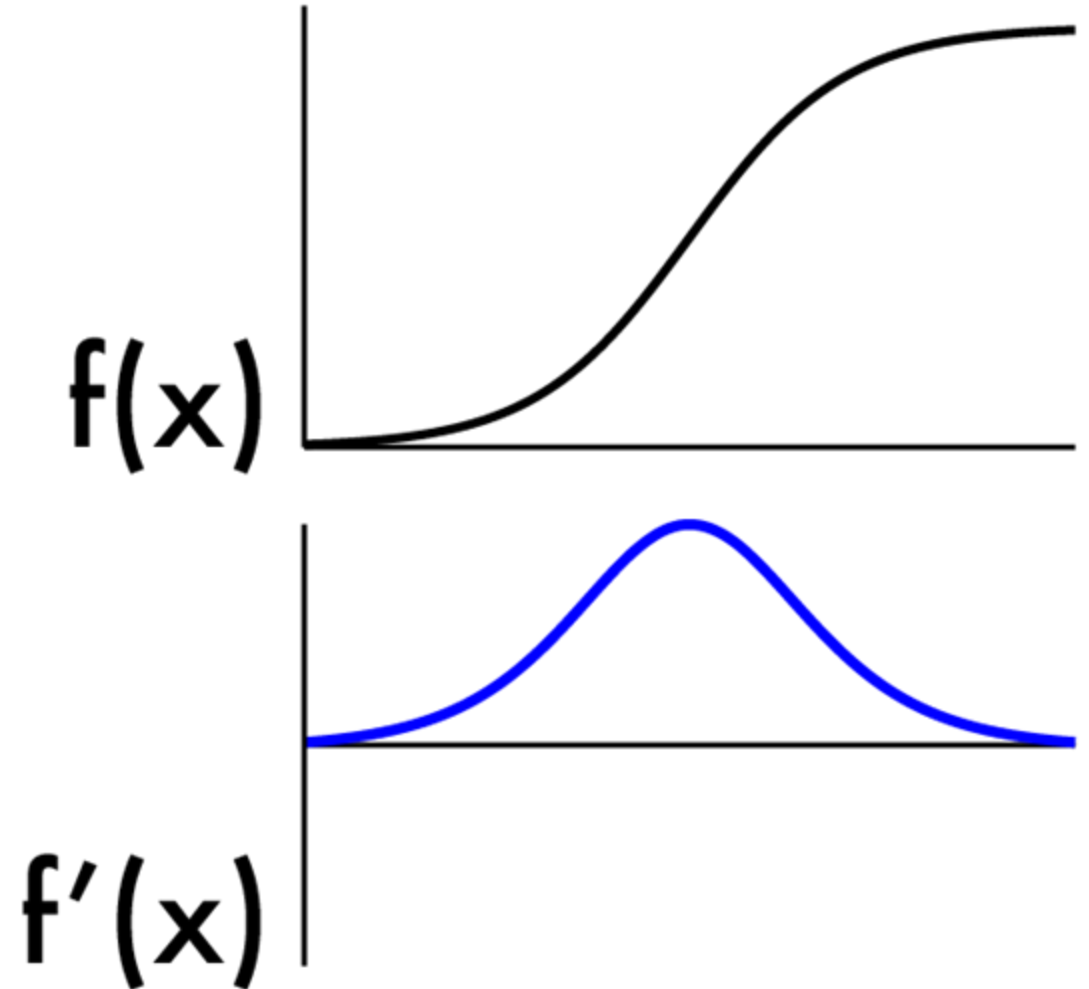


Image derivatives

- Recall:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

- We don't have an "actual" Function, must estimate
- Possibility: set $h = 1$
- What will that look like?

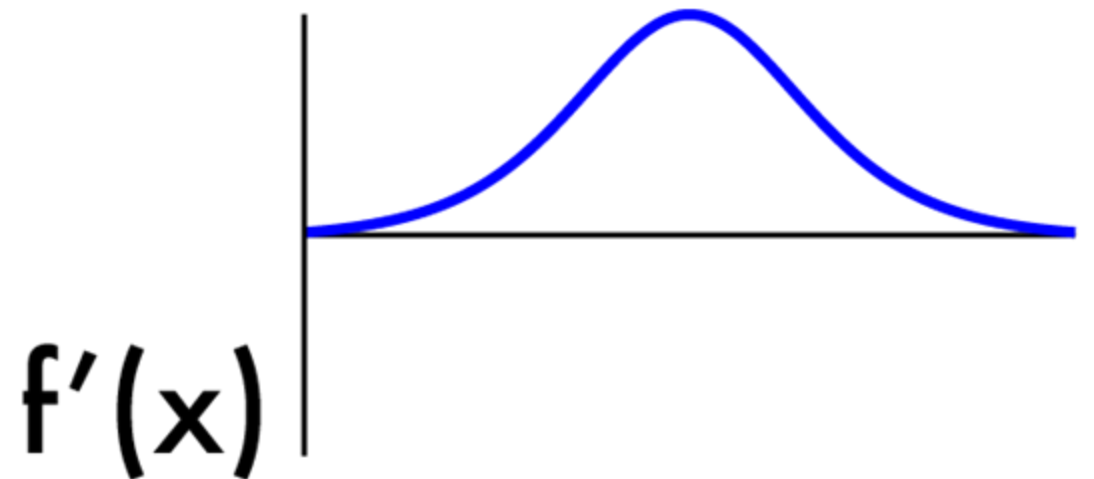
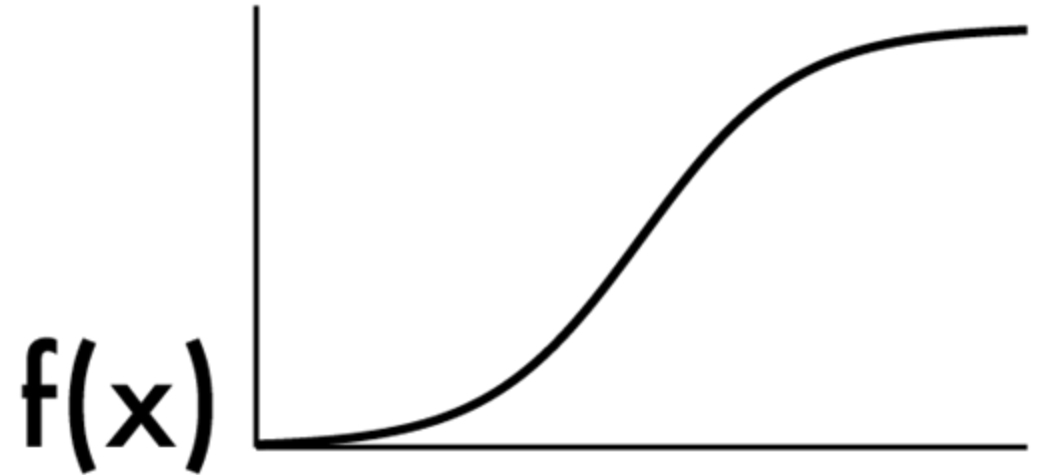
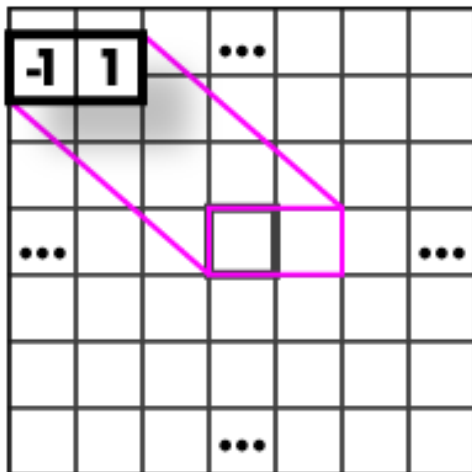


Image derivatives

- Recall:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

- We don't have an "actual" Function, must estimate
- Possibility: set $h = 2$
- What will that look like?

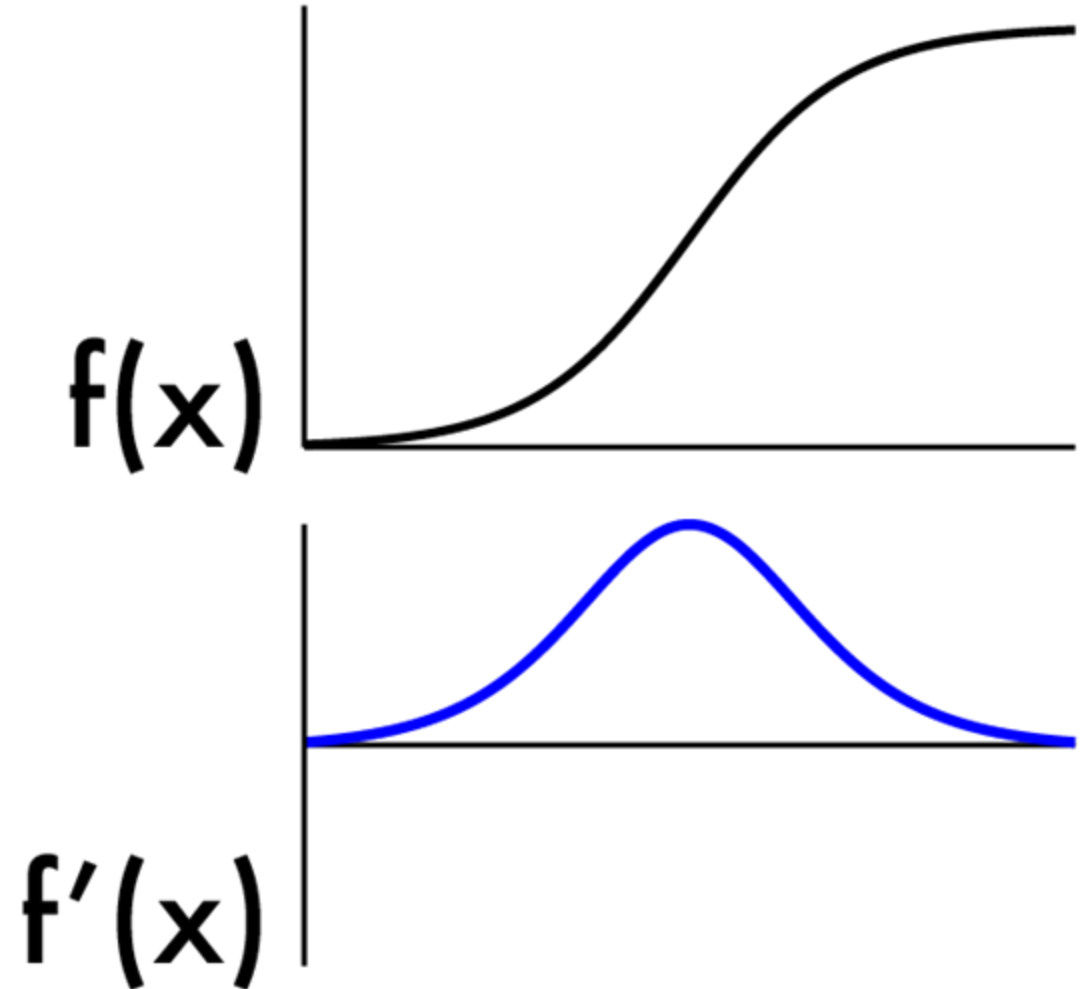
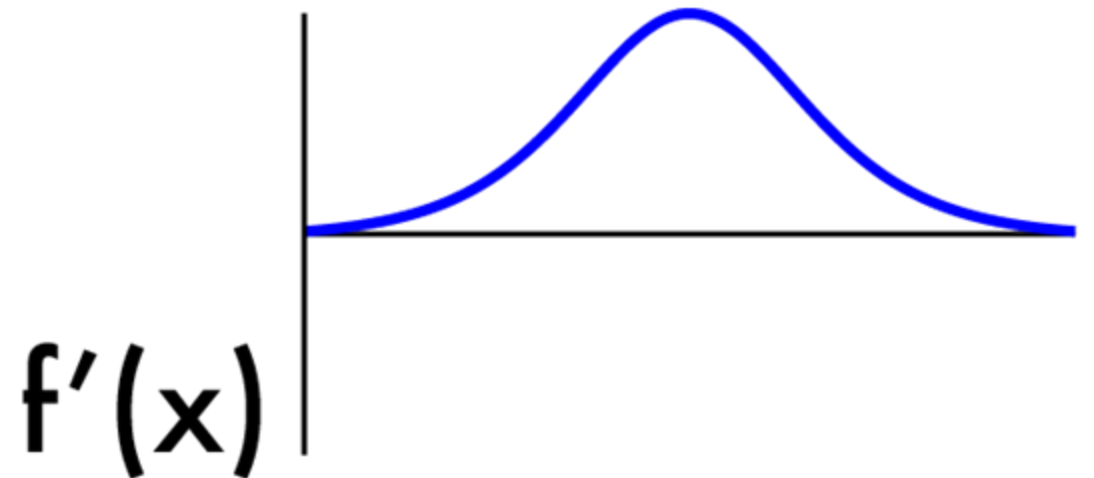
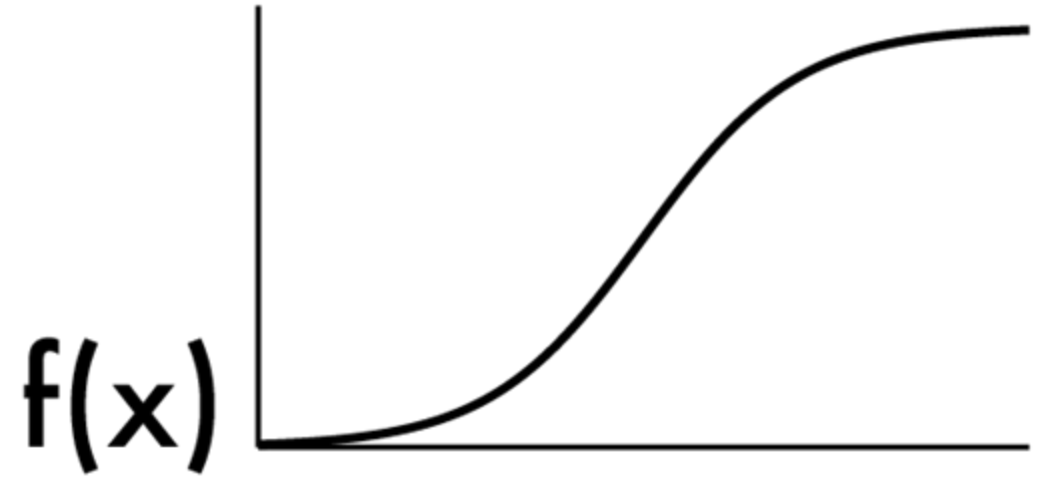
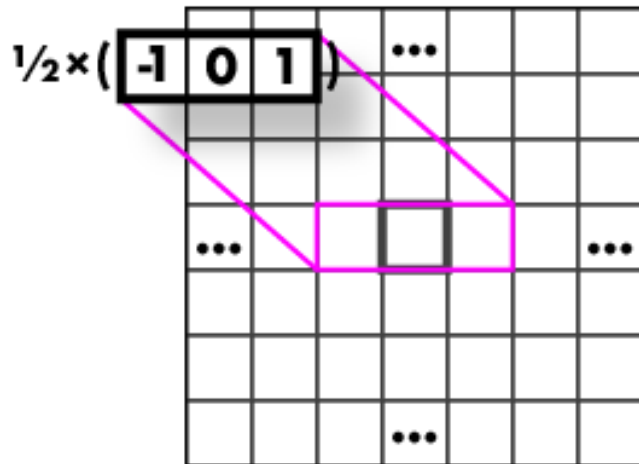


Image derivatives

- Recall:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

- We don't have an "actual" Function, must estimate
- Possibility: set $h = 2$
- What will that look like?



"X דוגמא לנגזרת הכי פשוטה – "ציר

$$\frac{\partial}{\partial x} f(i, j) \cong f(i, j) - f(i - 1, j)$$



 Mentimeter

Implement : $\begin{pmatrix} 1 & -1 \end{pmatrix}$
convolution with

 Mentimeter

"X דוגמא לנגזרת הכי פשוטה – "ציר

$$\frac{\partial}{\partial x} f(i, j) \cong f(i, j) - f(i - 1, j)$$



Implement : $\begin{pmatrix} 1 & -1 \end{pmatrix}$
convolution with

"דוגמא לנגזרת הכי פשוטה – "ציר

$$\frac{\partial}{\partial y} f(i, j) \cong f(i, j) - f(i, j - 1)$$

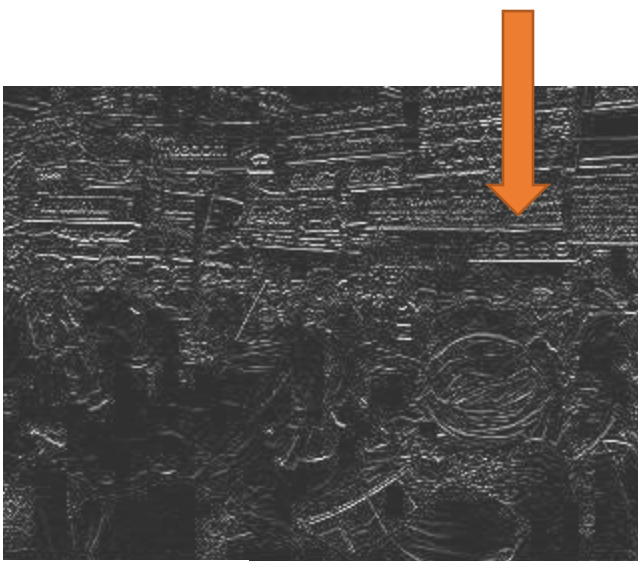


Implement :
convolution with

$$\begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

"דוגמא לנגזרת הכי פשוטה – "ציר

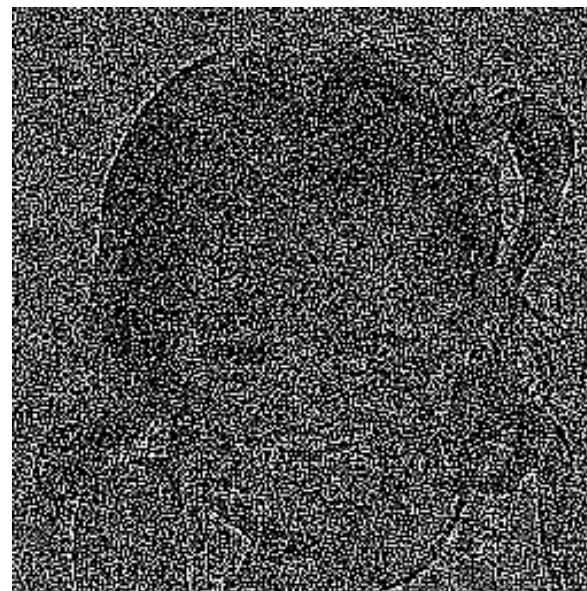
$$\frac{\partial}{\partial y} f(i, j) \cong f(i, j) - f(i, j - 1)$$



Implement :
convolution with

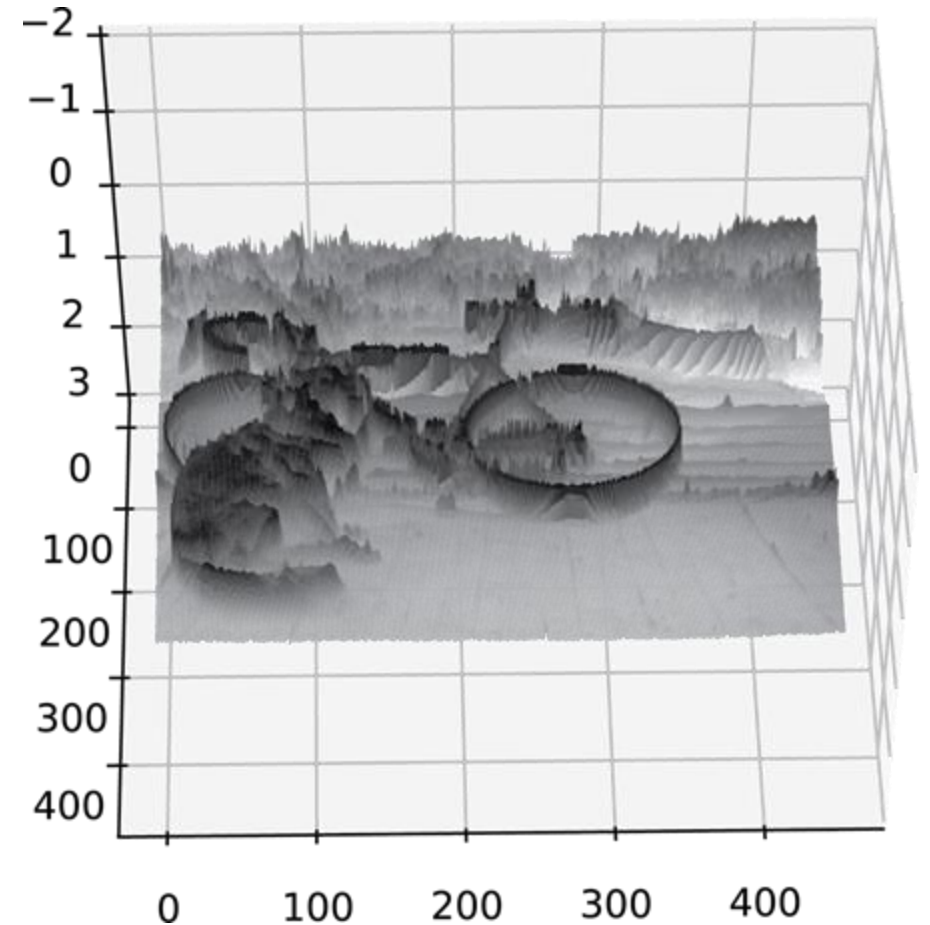
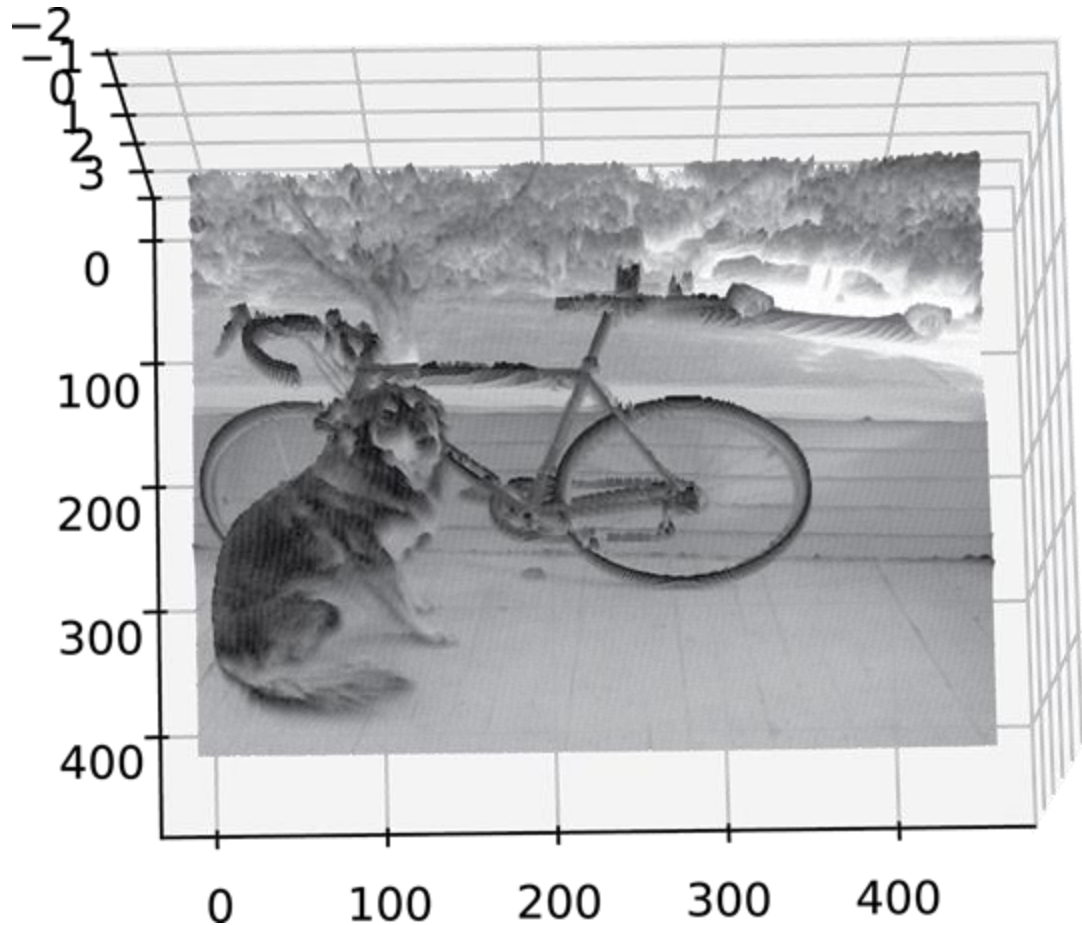
$$\begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

נגזרת ראשונה של תמונה רועשת

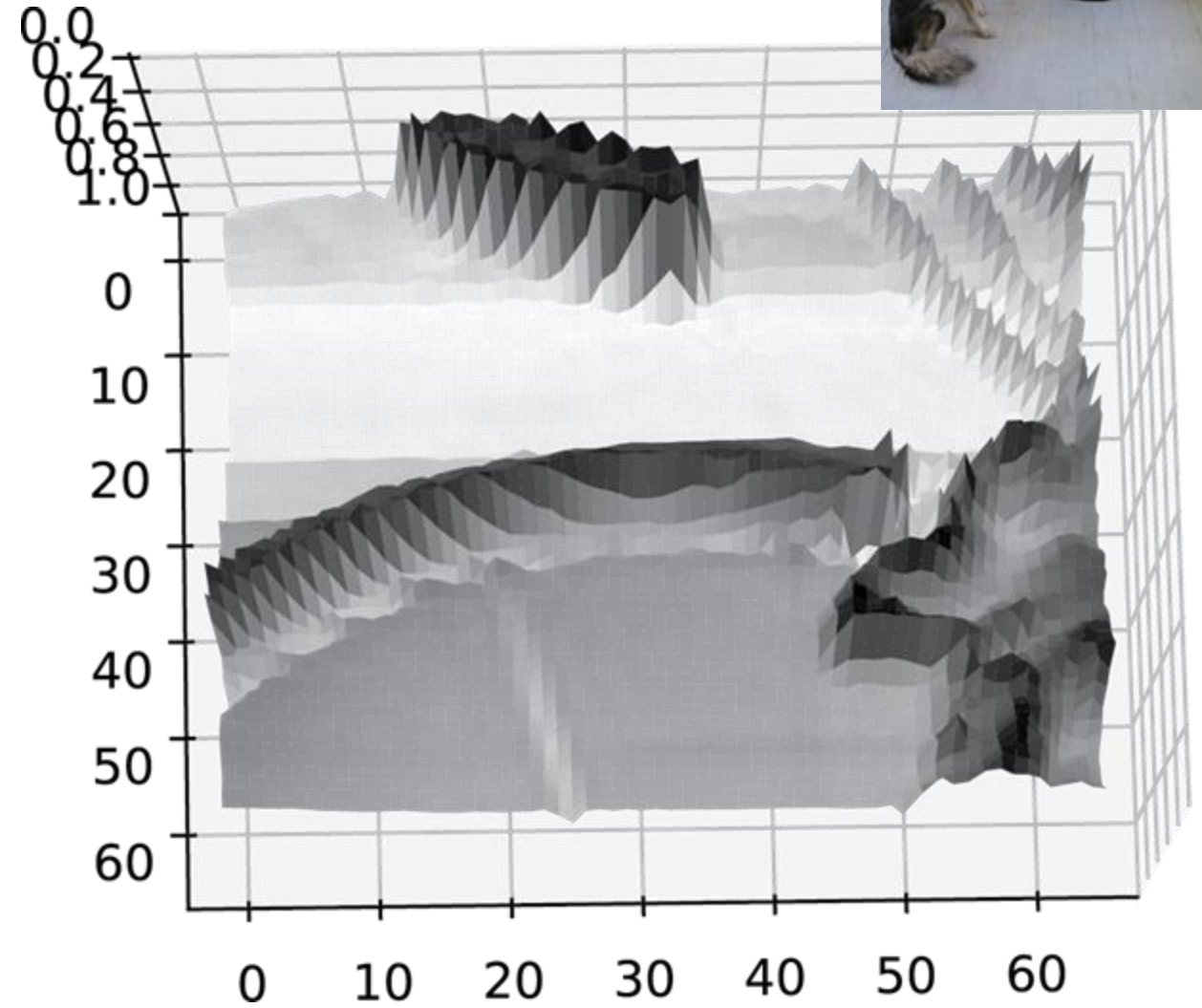
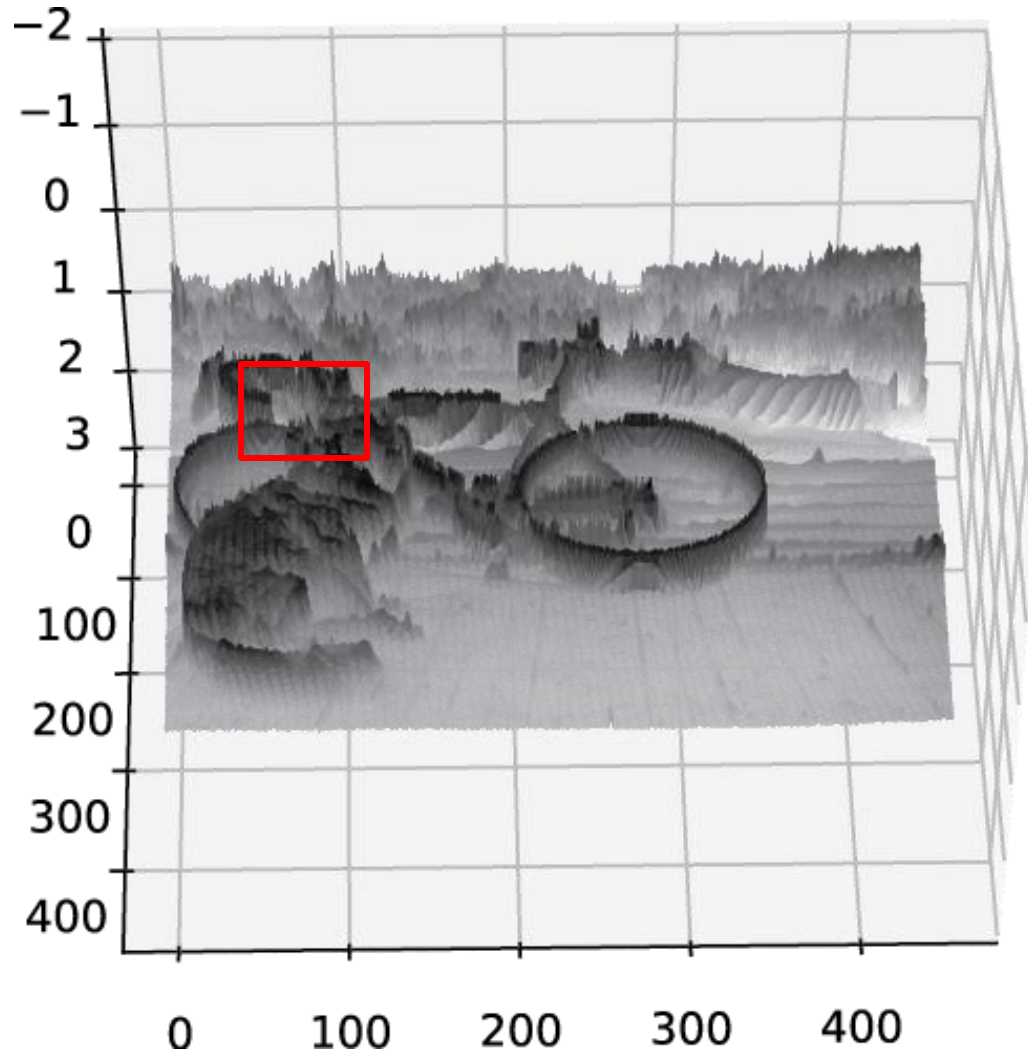


What's an edge?

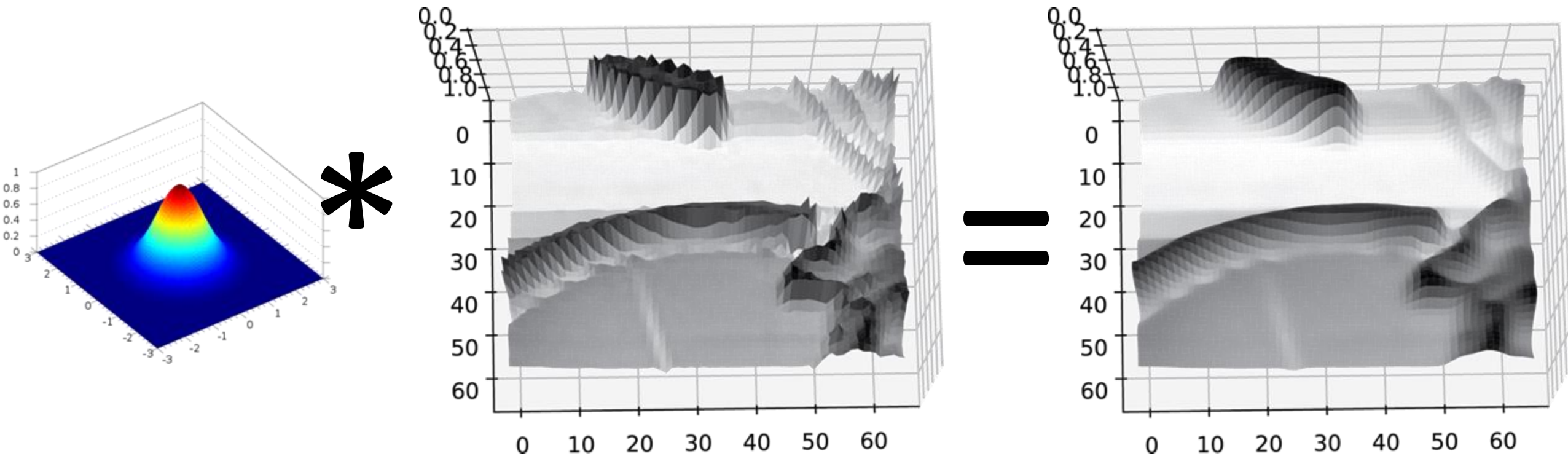
- Image is a function
- Edges are rapid changes in this function



Images are noisy!

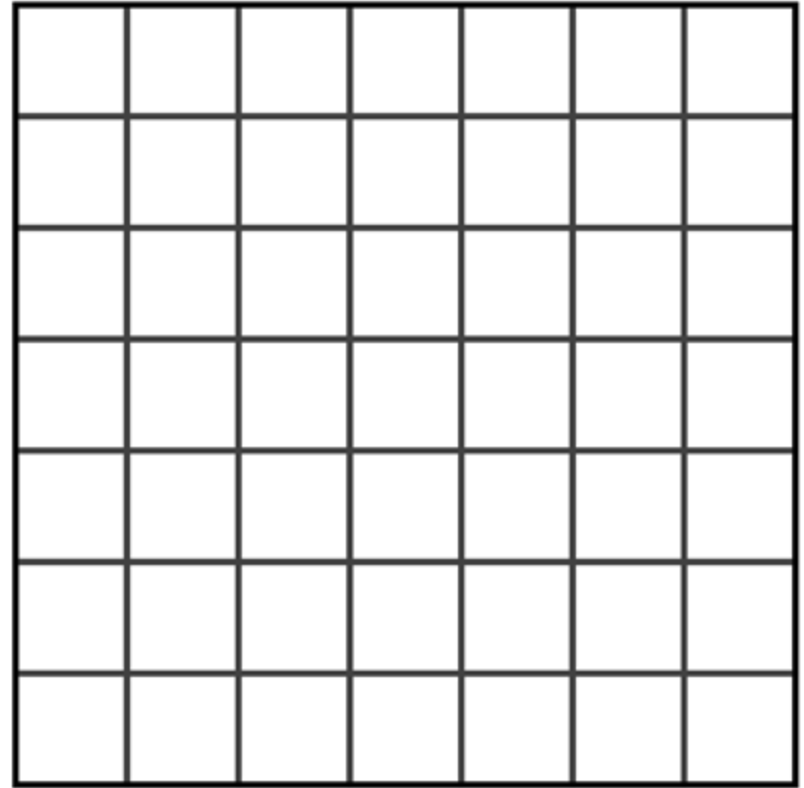


But we already know how to smooth



Smooth first, then derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right) *$$



Smooth first, then derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$

-1 0 1

1	2	1
2	4	2
1	2	1

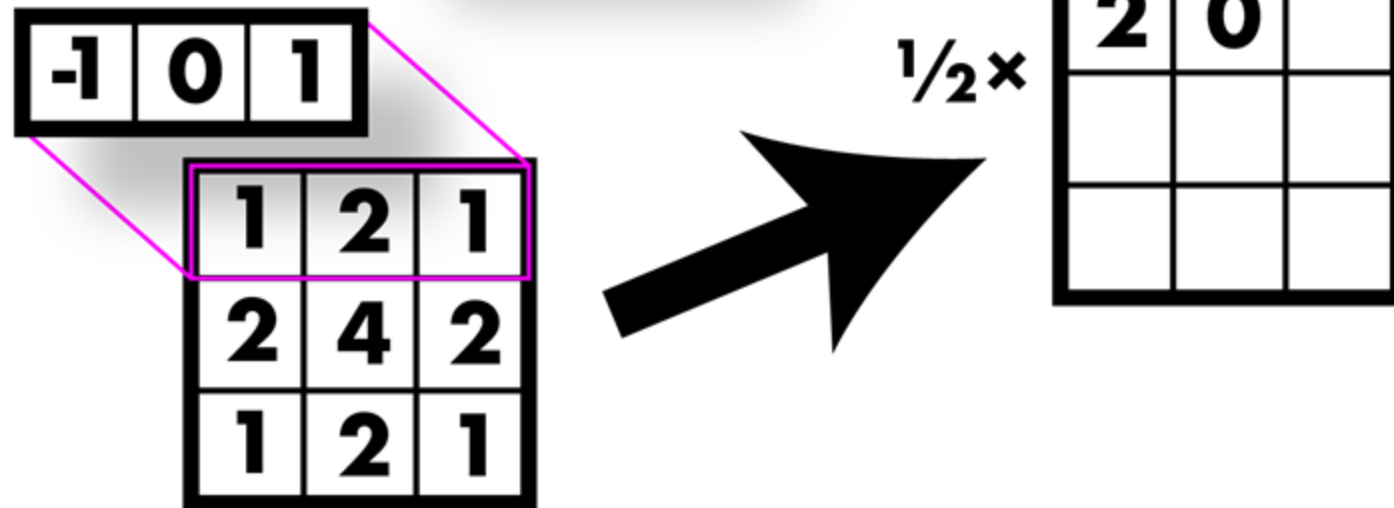


$\frac{1}{2} \times$

2		

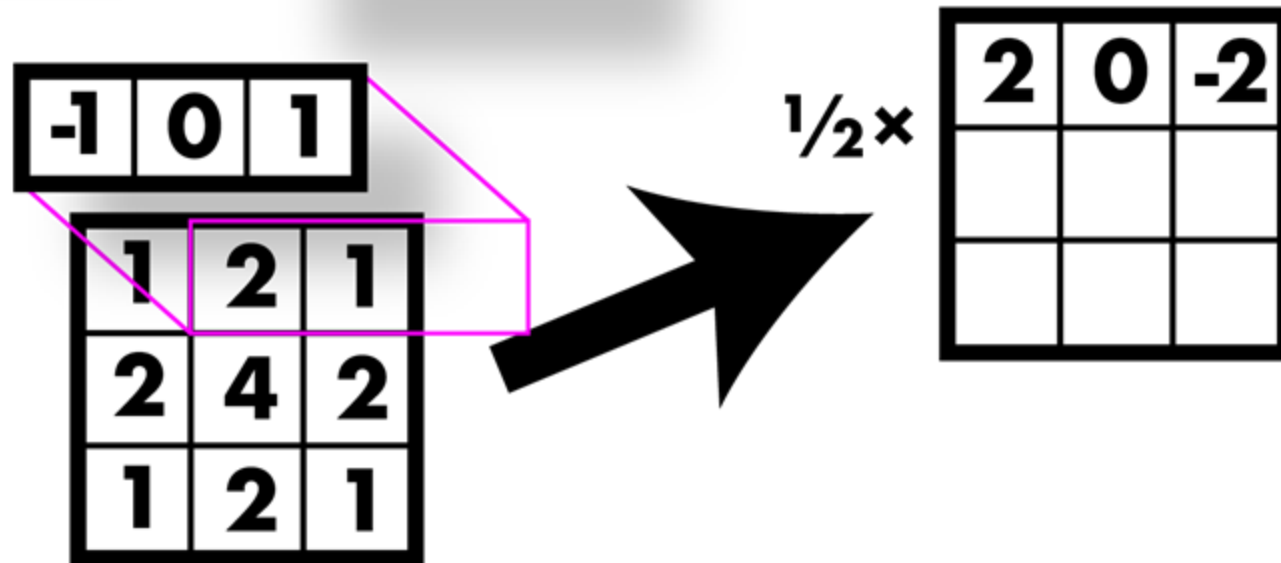
Smooth first, then derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



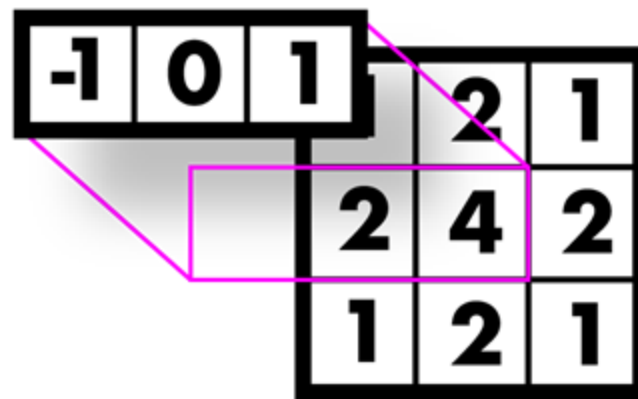
Smooth first, then derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



Smooth first, then derivative

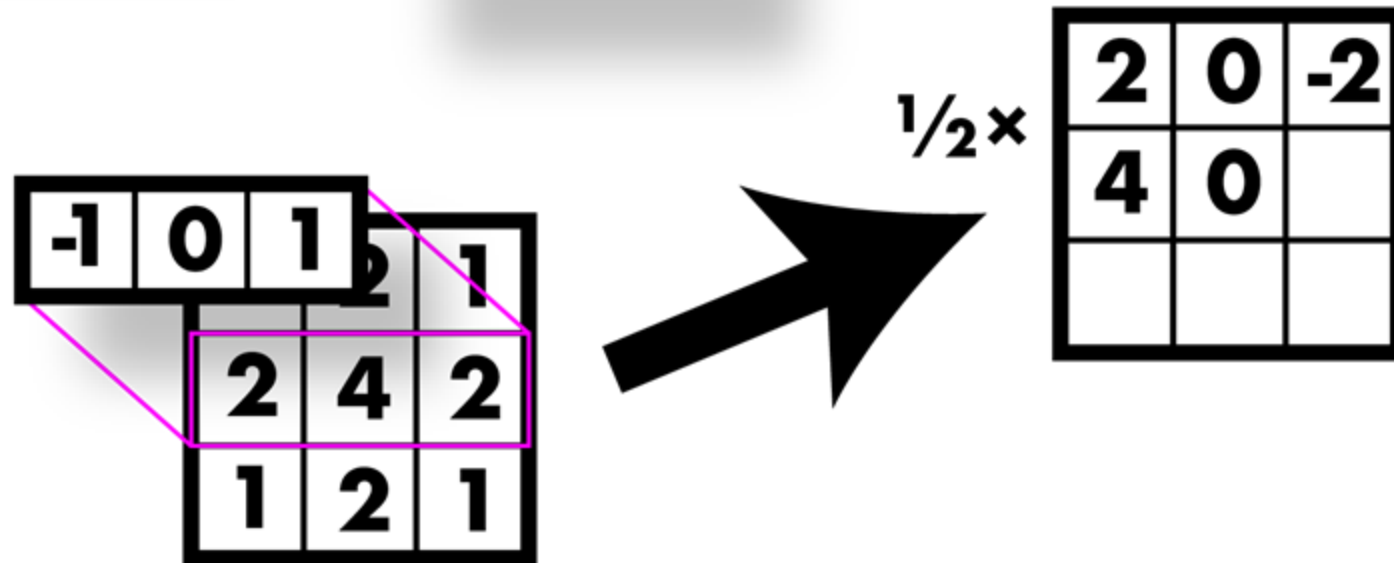
$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



$$\frac{1}{2} \times \begin{bmatrix} 2 & 0 & -2 \\ 4 & & \\ & & \end{bmatrix}$$

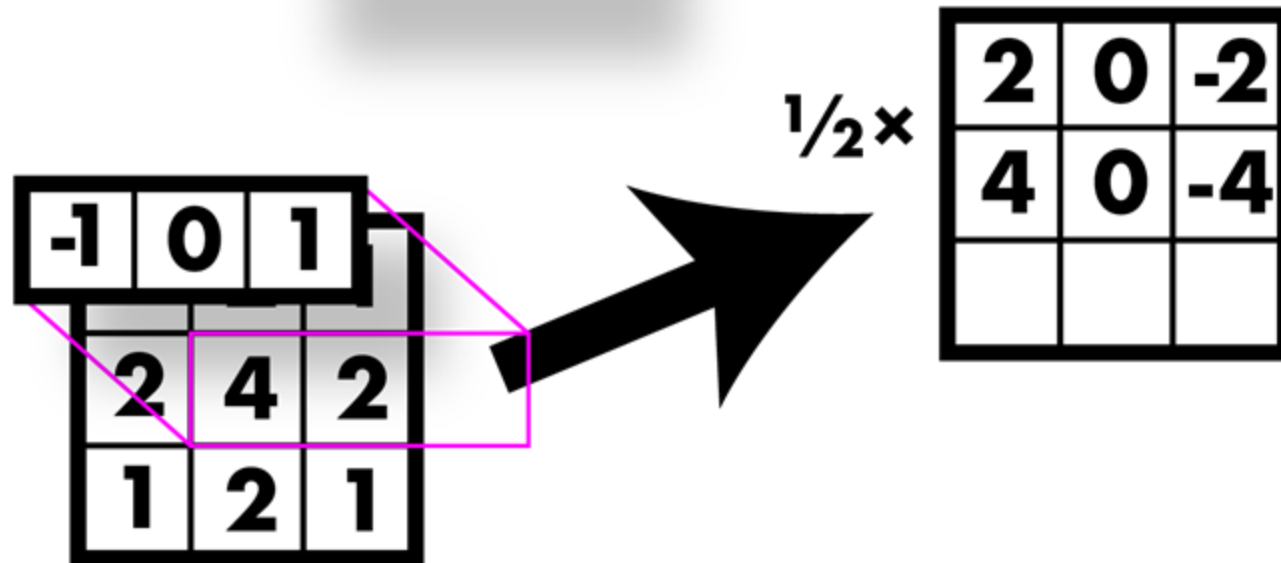
Smooth first, then derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



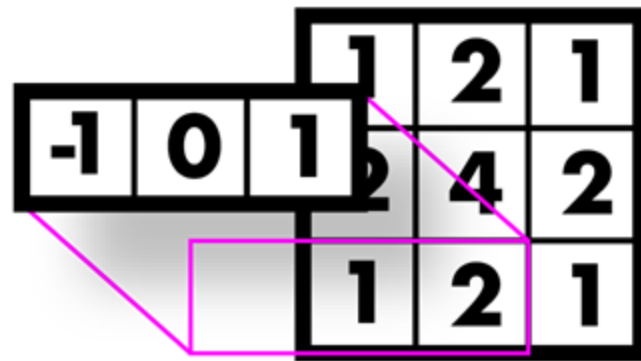
Smooth first, then derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



Smooth first, then derivative

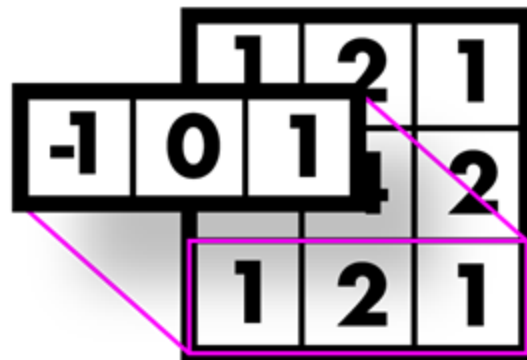
$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



$$\frac{1}{2} \times \begin{bmatrix} 2 & 0 & -2 \\ 4 & 0 & -4 \\ 2 & & \end{bmatrix}$$

Smooth first, then derivative

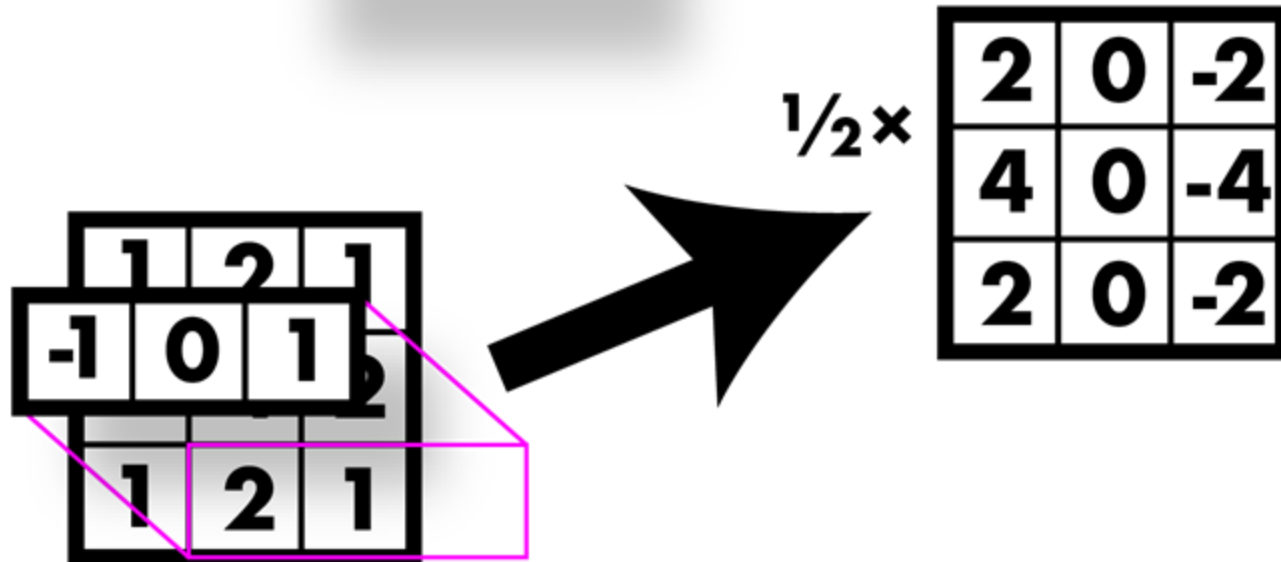
$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



$$\frac{1}{2} \times \begin{bmatrix} 2 & 0 & -2 \\ 4 & 0 & -4 \\ 2 & 0 & \end{bmatrix}$$

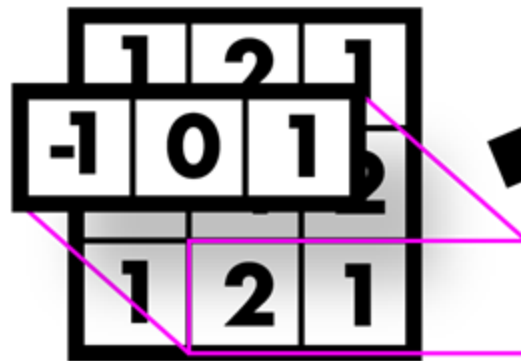
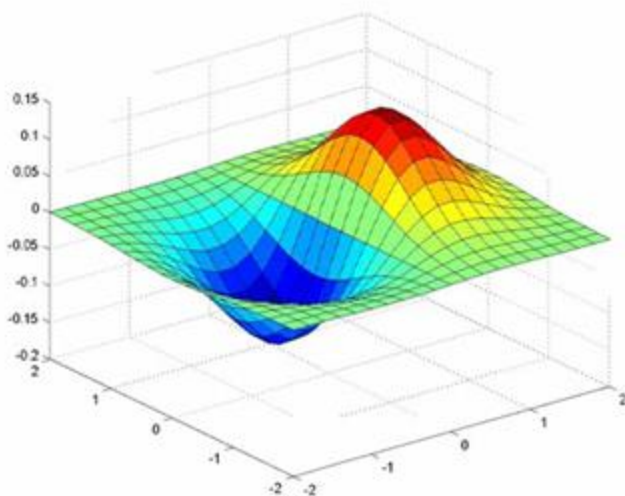
Smooth first, then derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



Sobel filter! Smooth & derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



1	0	-1
2	0	-2
1	0	-1

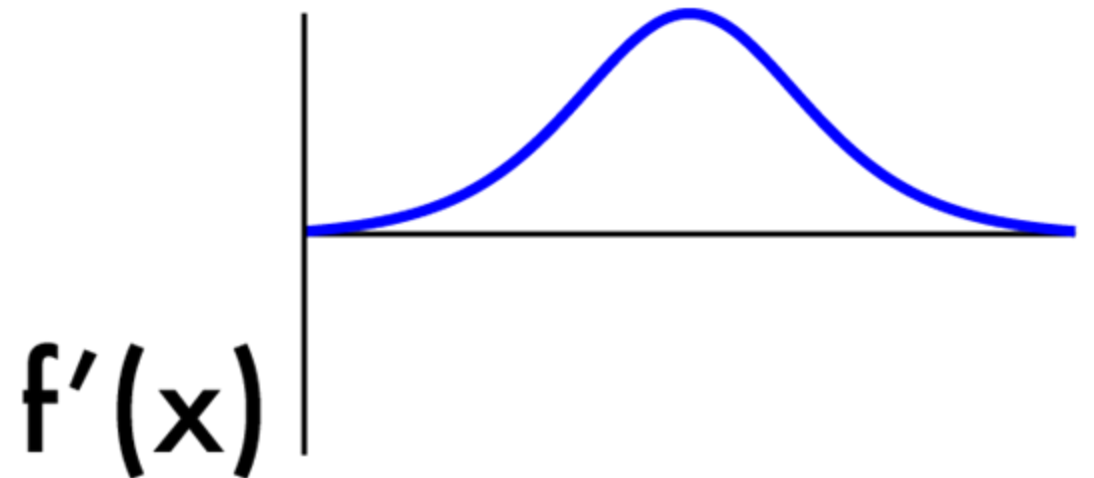
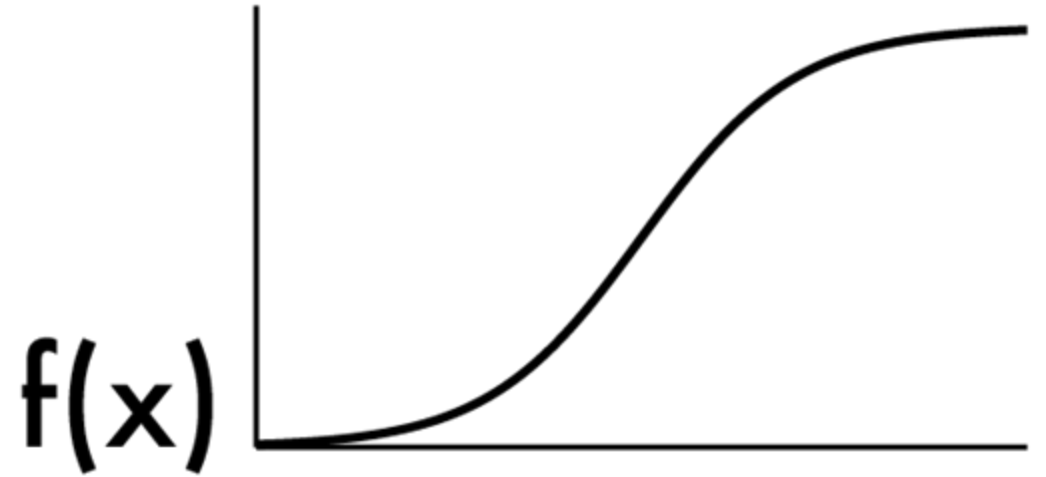
Image derivatives

- Recall:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

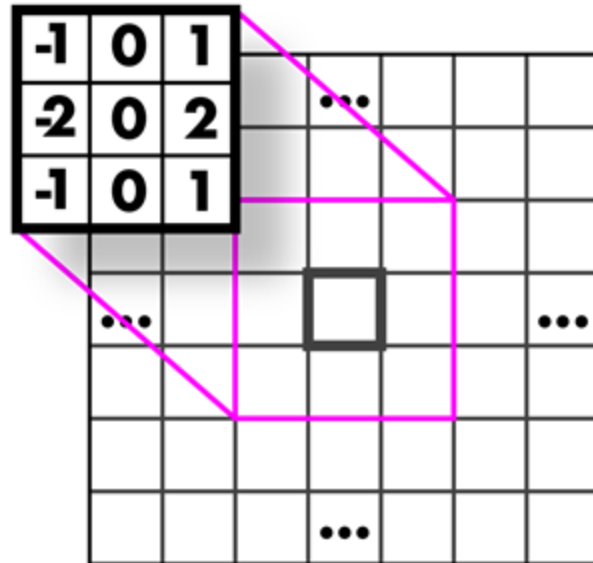
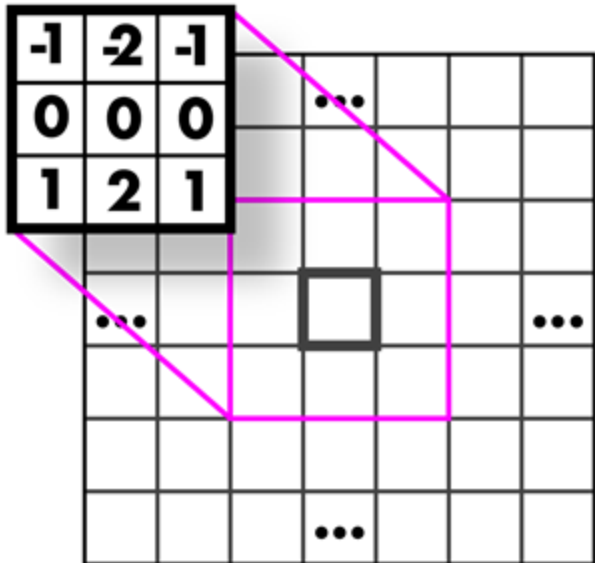
- Want smoothing too!

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$
$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

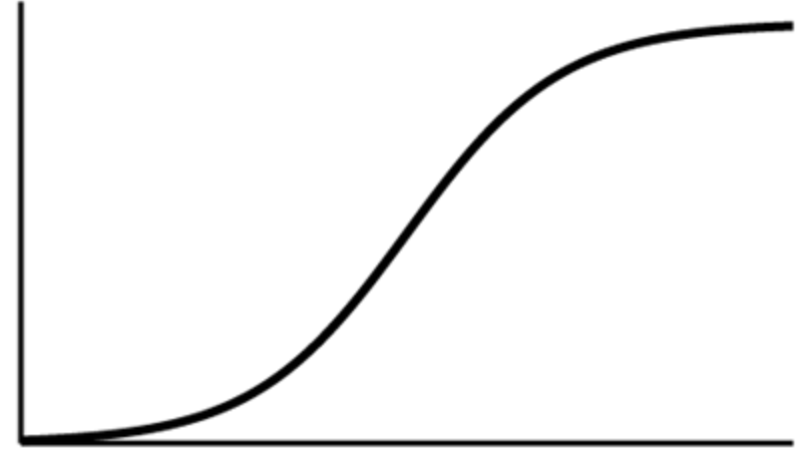


Finding edges

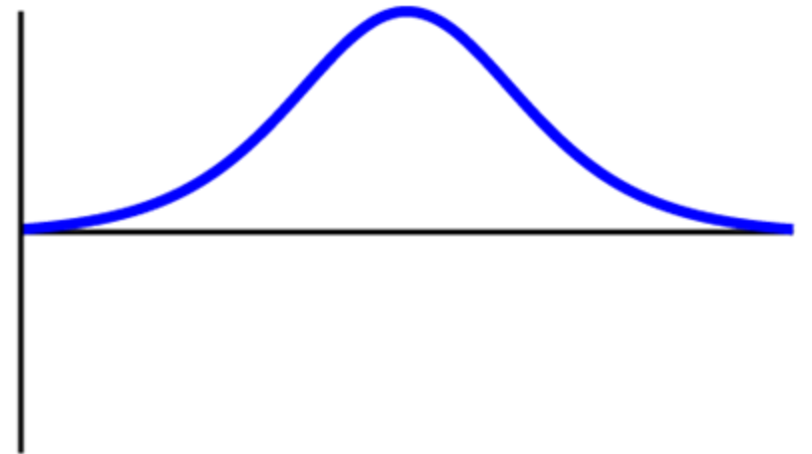
- Could take derivative
- Find high responses
- Sobel filters!
- But...



$f(x)$



$f'(x)$



Knowns as Sobel filters!

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

מה עושה הפילטר הבא ?

-1	-1	-1	-1	0	1
0	0	0	-1	0	1
1	1	1	-1	0	1

Prewitt

פילטר אשר מבצע החלקה פשוטה ביותר + נגזרת

-1	-1	-1	-1	0	1
0	0	0	-1	0	1
1	1	1	-1	0	1

Prewitt

מדדי איכות תמונה

ושימור קצוות PSNR, SSIM

Image Quality
Metrics

PSNR - Peak Signal-to-Noise Ratio

(MSE) מדד לאיכות תמונה המבוסס על **השגיאה הריבועית הממוצעת**

- משווה בין תמונת מקור לתמונה מעובדת/דחוסה

$$\text{MSE} = (1/mn) \times \sum [I(i,j) - K(i,j)]^2$$

$$\text{PSNR} = 20 \times \log_{10}(\text{MAX}/\sqrt{\text{MSE}})$$

MAX - ערך מקסימלי (255-bit לתמונות 8 ערך מקסימלי - MAX)

לדחיסה טובה **dB** ערכים טיפוסיים: **30-50**

קשה להבחין בהבדלים - **dB** מעל **40**

SSIM - Structural Similarity Index

- מדד המתחשב במבנה התמונה ובתפיסה האנושית משווה שלושה מרכיבים: **בהירות, ניגודיות ומבנה**
טווח ערכים: **0 עד 1** (1 = זהות מושלמת)

מבנה (Structure)

$$s(x,y) = (\sigma_{xy} + C_3) / (\sigma_x \sigma_y + C_3)$$

σ_{xy} = קו-וריאנס

ניגודיות (Contrast)

$$c(x,y) = (2\sigma_x \sigma_y + C_2) / (\sigma_x^2 + \sigma_y^2 + C_2)$$

σ = סטיית תקן

בהירות (Luminance)

$$l(x,y) = (2\mu_x \mu_y + C_1) / (\mu_x^2 + \mu_y^2 + C_1)$$

μ = ממוצע פיקסלים

$$SSIM(x,y) = l(x,y) \times c(x,y) \times s(x,y)$$

מדד שימור קצוות - Edge Preservation Index

מודד את **איכות שימור הקצוות** לאחר עיבוד התמונה

- חשוב במיוחד בפילטרים להפחתת רעש
gradient correlation מבוסס על

← Edge →

$$EPI = \sum(\nabla I_{orig} \cdot \nabla I_{proc}) / \sqrt{(\sum|\nabla I_{orig}|^2 \times \sum|\nabla I_{proc}|^2)}$$

פרשנות ערכים

שימור מצוין: $EPI \approx 1$

איבוד קצוות: $EPI < 0.9$

חישוב Gradient

$$\nabla I = (\partial I / \partial x, \partial I / \partial y)$$

Sobel/Prewitt

תרגיל סיום

cv2

```
grad_x = cv2.filter2D(layer, -1,  
                      np.array([[ -1,  0,  1],  
                                [-2,  0,  2],  
                                [-1,  0,  1]]))
```

```
grad_y = cv2.filter2D(layer, -1,  
                      np.array([[ -1, -2, -1],  
                                [ 0,  0,  0],  
                                [ 1,  2,  1]]))
```

שימושים פשוטים של נגזרת ראשונה

תרגיל כיתה

- צרו (בקוד, לא להשתמש בצייר..) את התמונה המלבן הבאה

- – נראה כמו 8.

- גודל מינימלי של התמונה 200X200

- כתבו פונקציה אשר

- מקבלת 2 מטריצות: תמונה ופילטר

- ומחזירה את התמונה אחרי פעולת הקונבולציה בין התמונה לפילטר

- אפשר להשתמש בפונקציה `cv2.filter2D` בלבד

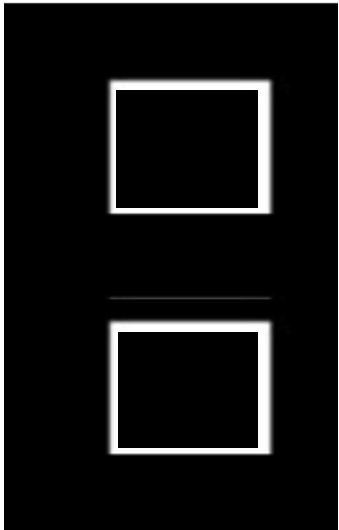
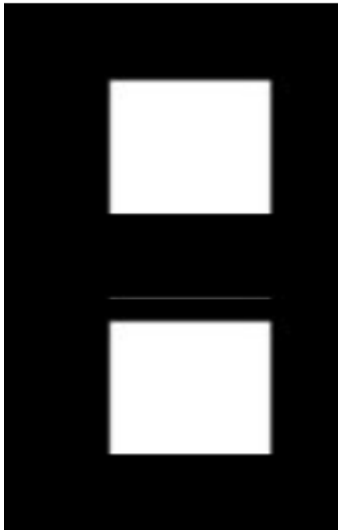
- כלומר, בעזרת שימוש בנגזרות ופונקציית הקונבולציה לעיל מייצרת את

תמונת המסגרת של הקוביות הלבנות.

- פלט התוכנית: שתי חלונות (או אחד מפוצל) של המלבן המקורי

תזכורת: פילטר = kernal = mask. זה בדו"כ אותם מילים לאותו דבר

והמסגרת.



שימושים פשוטים של נגזרת ראשונה

תרגיל 1ב



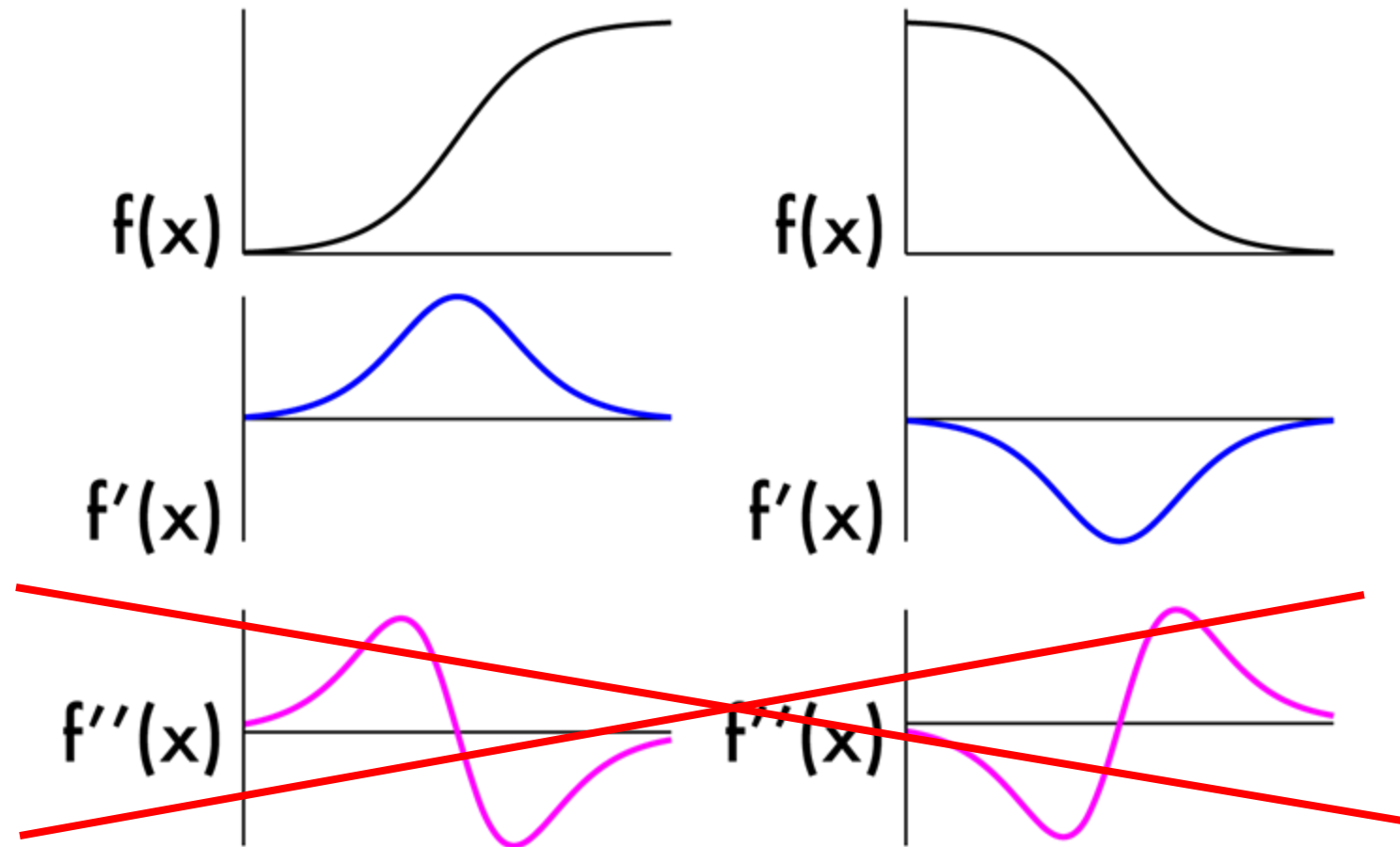
- יצרו את התמונה המלבן הבאה:
- גודל מינימלי של התמונה 200X200
- הוסיפו באופן אקראי ל-5% מהפיקסלים רעש גאוסיאני לתמונה.
- רמז `np.random.normal / randint`
- כתבו תוכנית באשר בעזרת שימוש בנגזרות מייצרת את תמונת המסגרת של המלבן כמייטב יכולתכם.
- תוך שימוש ה `Numpy` בלבד (ניתן לקרוא את התמונות עם `opencv`) פלט התוכנית: שתי חלונות (או אחד מפוצל) של המלבן המקורי והמסגרת.
- פלט התוכנית: שתי חלונות (או אחד מפוצל) של המלבן המקורי והמסגרת.

עיבוד תמונה – שיעור 4

2021

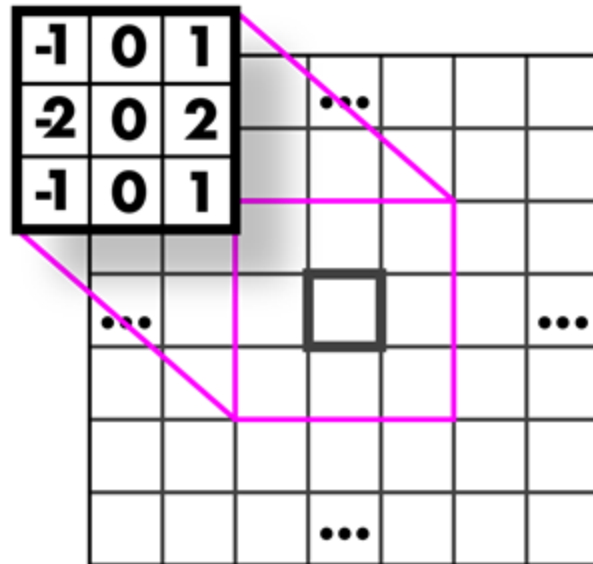
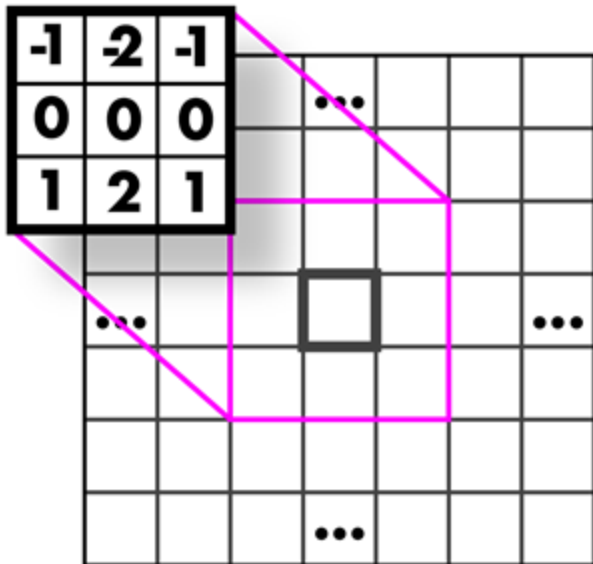
Another approach: gradient magnitude

- Don't need 2nd derivatives
- Just use magnitude of gradient



Another approach: gradient magnitude

- Don't need 2nd derivatives
- Just use magnitude of gradient
- Are we done? No!





What we really want: line drawing



```
= import matplotlib.pyplot as plt

img = cv2.imread('pic1.PNG', 0)

filter_blured = cv2.GaussianBlur(img, (7, 7), 0)
alpha = 5
sharpened = img + alpha * (img - filter_blured)

images = [img, filter_blured, sharpened]
titles = ["Orig image", "GaussianBlur", "Sharpened"]

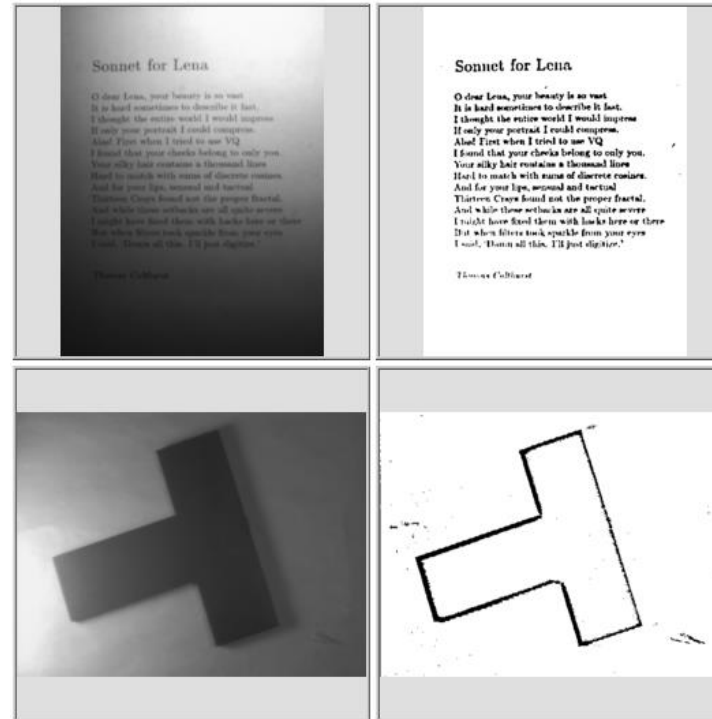
= for i in range(3):
    plt.subplot(3, 1, i+1), plt.imshow(images[i], 'gray')
    plt.title(titles[i])
```

עיבוד תמונה – שיעור 4

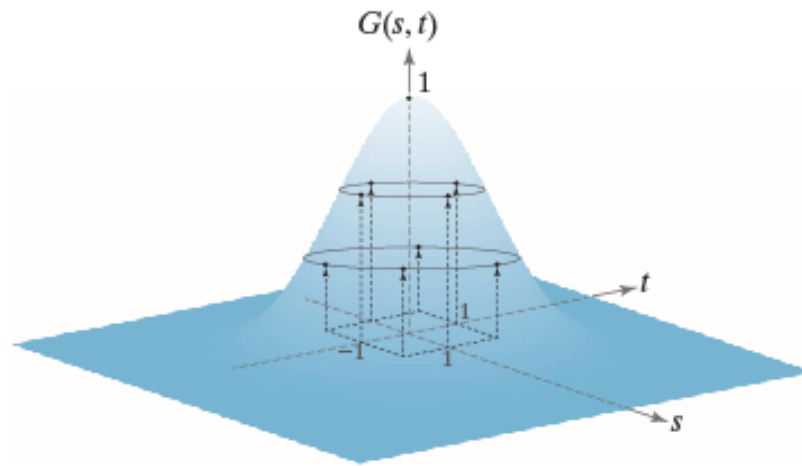
- 1. חזרה
- 2. point processing – "פעולות נקודה"
 - 1. עוד שימושים של הסטוגרמות
 - 2. סף – Thresholding
- 3. Neighborhood processing
- 4. Convolutions
 - 1. Zero padding
- 5. Model of noise
- 6. Classic filters
 - 1. מיצוע – Blur
 - 2. מיצוע גאוסייאני – Gaussian filter
- 7. Sharpening – חידוד תמונה
- 8. Adaptive threshold

Adaptive Thresholding

- Thresholding is space variant.
- How can we choose the local threshold values?
- In the previous section, we used one global value as a threshold. But this might not be good in all cases, e.g. if an image has different lighting conditions in different areas. In that case, adaptive thresholding can help. Here, the algorithm determines the threshold for a pixel based on a small region around it. So we get different thresholds for different regions of the same image which gives better results for images with varying illumination.



cv.ADAPTIVE_THRESH_GAUSSIAN_C



$$\frac{1}{4.8976} \times$$

0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679

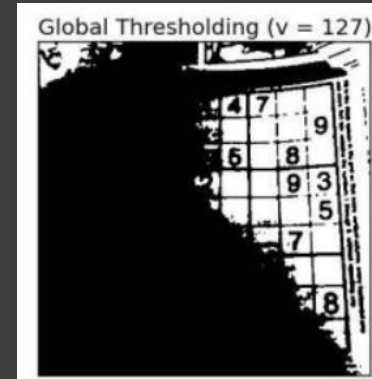
```

img = cv.imread('sudoku.png',0)
img = cv.medianBlur(img,5)
ret,th1 =
cv.threshold(img,127,255,cv.THRESH_
BINARY)

th2 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_MEAN_C,\
cv.THRESH_BINARY,11,2)

th3 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_GAUSSIAN_C,\
cv.THRESH_BINARY,11,2)

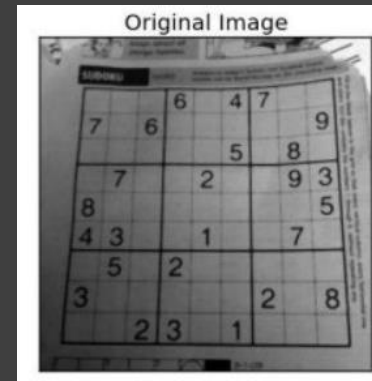
```



```
img = cv.imread('sudoku.png',0)
img = cv.medianBlur(img,5)
ret,th1 =
cv.threshold(img,127,255,cv.THRESH_
BINARY)

th2 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_MEAN_C,\
cv.THRESH_BINARY,11,2)

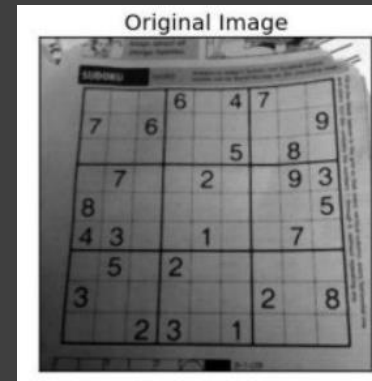
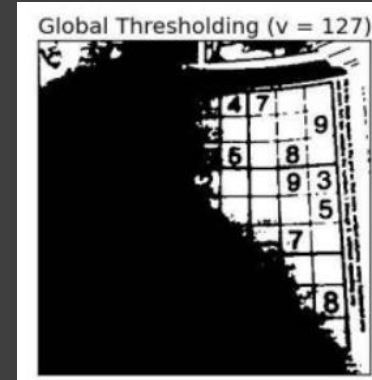
th3 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_GAUSSIAN_C,\
cv.THRESH_BINARY,11,2)
```



```
img = cv.imread('sudoku.png',0)
img = cv.medianBlur(img,5)
ret,th1 =
cv.threshold(img,127,255,cv.THRESH_
BINARY)
```

```
th2 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_MEAN_C,\
cv.THRESH_BINARY,11,2)
```

```
th3 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_GAUSSIAN_C,\
cv.THRESH_BINARY,11,2)
```



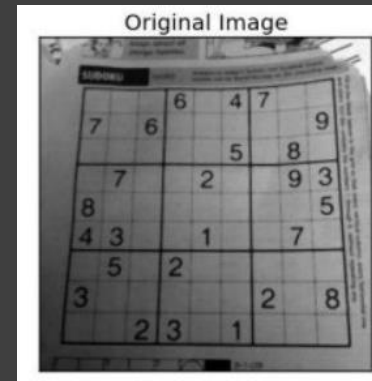
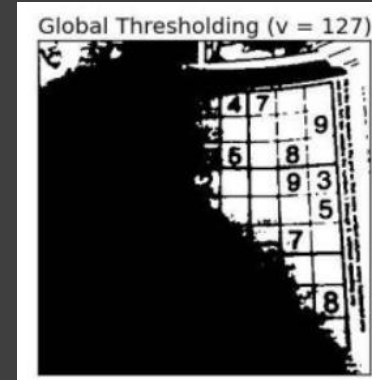
```

img = cv.imread('sudoku.png',0)
img = cv.medianBlur(img,5)
ret,th1 =
cv.threshold(img,127,255,cv.THRESH_
BINARY)

th2 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_MEAN_C,\
cv.THRESH_BINARY,11,2)

th3 =
cv.adaptiveThreshold(img,255,cv.ADA
PTIVE_THRESH_GAUSSIAN_C,\
cv.THRESH_BINARY,11,2)

```



שאלה 1

צלם תמונה, טשטש אותה בעזרת פילטר מתאים לשלוש רמות טשטוש שונות.

וחשב את האנטרופיה לכל אחת מהתמונות המטושטשות והמקורית.

התוכנית תציג מסך עם ארבע תמונות-- בתצורת 2×2 את התמונה המקורית עם שלוש רמות טישטש כאשר בכותרת את ערך האינטרופיה

שאלה 2

כתוב תוכנית אשר מוסיפה רעש גאוסייאני לתמונה.

מציגה את התמונה המקורית וליידה אתה התמונה בתוספת רעש גאוסייאני